

The Alberta Buck - Notes (v2)

Private Bearer Instruments on an Identity-Aware Demurrage Currency

Perry Kundert

2026-06-04



For forty years cryptographers have tried to build digital cash and failed. Chaum's DigiCash (1990s) had the mathematics but went bankrupt for want of merchant adoption. Bitcoin gave us digital scarcity but no native bearer-paper form – Casascius coins and BIP-38 paper wallets are aftermarket wrappers, not first-class instruments. Central bank digital currencies threaten the opposite: digital surveillance with no bearer privacy at all.

The Alberta Buck identity, reserve, and demurrage machinery – combined with one new primitive, a global commitment pool – finally permits a native digital-private bearer instrument. This document specifies that primitive.

A **BUCK Note** is an on-chain commitment whose opening authorises a private mint to a registered identity. The same commitment can live only on chain (a software-only transfer between two of a user's own private accounts), travel between parties as a digital message, or be printed on paper as a QR-encoded slip – all three surfaces sharing the same cryptography and the same on-chain footprint, indistinguishable in use from a \$20 bill while being cryptographically impossible to counterfeit.

Notes defeat *bulk on-chain traceability* – KYC-provider subpoenas, chain-analytics fingerprinting, omnibus surveillance – while preserving *bilateral* Identity disclosure: the two named parties to any specific note always learn each other's M, and either is compellable on warrant. This is stronger compliance than physical cash provides today, stronger privacy than any CBDC has ever proposed, and the first design where a private bearer instrument is a native, first-class primitive of the monetary system rather than an aftermarket wrapper. (PDF, Text, Walkthrough)

Contents

1	The Cash Paradox	5
1.1	Why Forty Years of Trying	5
1.2	BUCK Notes at a Glance	7
2	The Core Primitive: One Global Commitment Pool	8
2.1	The Note Contract State	8
2.2	The Note Commitment	8
2.3	The Note Nullifier	9
2.4	Lifecycle	9
3	A Note is a Deferred Approve Handshake	11
3.1	The Regular On-Chain Transfer	11
3.2	The Note is the Same Handshake, Deferred	11
3.3	Mutual Decryptability: The BUCK Identity Invariant	12
3.4	The Non-Deniable-Receipt Invariant	13
3.5	What the On-Chain Observer Sees	14
3.6	Why This Framing Matters	15
4	The Surface of a Note: Chain, Message, Paper	16
4.1	Chain-Only Notes	16
4.2	Message Notes	16
4.3	Paper Notes	16
4.4	Counterfeiting is Cryptographically Impossible	17
4.5	Wallet-App Status Codes	17
5	Note Flavors: Depositor Binding x Issuer Visibility	18
5.1	Flavor A2: Addressed Cheque, Private Issuer	18
5.2	Flavor A1: Addressed Cheque, Public Issuer	19
5.3	Flavor B1: Bearer Cheque, Public Issuer	19
5.4	Comparison of the Three Flavors	20
6	The Interaction Matrix: Note Flavor x Depositor Account Mode	21
7	Integration with On-Chain Transfers	23
7.1	The Pool is a Regular Account	23
7.2	Regular Transfer vs. A-Flavor Note	23
7.3	Regular Transfer vs. B-Flavor Note	24
7.4	When to Use Which	24
7.5	Cost and UX Trade-Offs	24
7.6	Why A2 Costs More: Anonymity Is the Cost	24
7.7	Composability	25
8	Demurrage at the Pool Level	26
8.1	Reminder: How BUCK Demurrage Works	26
8.2	The Pool Account's Demurrage	26
8.3	The Age-Preserving Redemption	27
8.4	Donations Integrate Smoothly	27

8.5	Conservation is Automatic, and Tight	28
8.6	Numeric Example	28
8.7	Who Bears the Demurrage, Really	28
9	Demurrage Solves the Lost-Cash Problem	30
9.1	The Hidden Inflation of Lost Cash	30
9.2	What BUCK Demurrage Does For Lost Notes	30
9.3	The Symmetry With On-Chain Balances	31
10	Long-Term Holding Safety: Addressed vs. Bearer	32
10.1	What the Issuer Knows	32
10.2	A-Flavors (Addressed): Cryptographically Safe, and Recoverable	32
10.2.1	Identity-Pinning and Recovery	33
10.3	B1 (Bearer): Civil-Trust, Not Cryptographic	34
10.4	Summary: Which Flavors Can Be Saved?	34
11	Physical Security Model	36
11.1	Layered Defense	36
11.2	Recommended Manufacturing Profile	36
12	Use Cases	38
12.1	Walking-Around Money	38
12.2	Cross-Border Remittances	38
12.3	Tip Jars and Charitable Donations	39
12.4	Cold Storage and Disaster Preparedness	39
12.5	Estate Planning	40
12.6	Gift Cards Without Issuer Risk	40
12.7	Tax-Compliant Compensation	40
12.8	Underbanked and Cash-Economy Inclusion	40
13	Privacy: Defeating Mass Harvest, Preserving Bilateral Disclosure	42
13.1	What the On-Chain Observer Loses	42
13.2	What Bilateral Parties Keep	42
13.3	The Mass-Harvest Threat Model	42
13.4	The Regulatory Audit Path	43
13.5	What Compliance Looks Like	43
13.6	The CBDC Comparison	44
14	Comparison With Prior Art	45
14.1	Chaum's E-Cash (1982-1998)	45
14.2	Brands' Offline Divisible E-Cash (1993)	45
14.3	Casascius Coins (2011-2013)	45
14.4	BIP-38 Paper Wallets	46
14.5	Zerocash / Zcash (2014-)	46
14.6	Aztec / Zk.Money	46
14.7	Tornado Cash and Privacy Pools	47
14.8	Comparison Summary	47
14.9	Comparison: Cash, CBDC, BUCK Notes	47

15 Required Identity Cryptography	49
15.1 Components Already Present	49
15.2 New Primitive: The Note Commitment Hash	49
15.3 New Primitive: The Nullifier PRF	50
15.4 Encryption at Mint – Under the Recipient’s Receiving Key (A2)	50
15.5 Deposit Gate (A1/A2) – One Folded Proof	50
15.6 Deposit Gate (B1) – Depositor Binding (the A2 dual)	51
15.7 Summary: Cryptographic Components	51
16 The SNARK Circuits	52
16.1 Mint Circuit (<code>circuits/mint_batch.circom</code>)	52
16.2 Spend: Note-Opening + the Identity-M Deposit Gate	53
16.3 Circuit Engineering Budget	54
16.4 On-Chain Gas (measured end-to-end)	55
16.5 Common Circuit Gadgets	55
17 What This Architecture Rules Out: Pure-Bearer Cryptographic Notes	57
18 Security Assumptions and Theorems Used	58
18.1 Demurrage Conservation as a Contract Invariant	58
19 Open Questions	59
19.1 Donation Incidence	59
19.2 Stranded Value on Lost Notes	59
19.3 Integer-Arithmetic Rounding	59
19.4 Predicate Programmability vs. Circuit Simplicity	59
19.5 Serialized B1 Sub-Notes (forward design)	59
19.6 Universal Spend Circuit (resolved for the note opening only)	60
19.7 Receiving Key, Still Open	60
19.8 Regulatory Interaction with the Carrying Transfer	60
20 Summary	61

1 The Cash Paradox

Physical cash is simultaneously irreplaceable and impossible to digitize.

It is irreplaceable because it does things no digital instrument has ever matched. A \$20 bill works without a network, a battery, a bank, an ID check, or a counterparty to your transaction. It clears instantly, finally, and without fees. It is the single most successful monetary instrument in human history – so successful that roughly half the value of US currency in circulation is held overseas as a global store of value, in defiance of every theoretical reason it shouldn't be.¹

It is impossible to digitize because every attempt has reproduced the surveillance properties of bank money instead of the privacy properties of cash. Modern card networks log every transaction. Bank wire transfers require both parties to be identified. Stablecoins record every transfer publicly on chain. Central bank digital currencies, where they exist or are being designed (China's e-CNY, Europe's digital euro proposals), are explicitly programmable surveillance tools – the central bank can see, restrict, expire, or geofence any holder's balance at will.²

The closest existing analogues to digital cash are aftermarket bearer wrappers around cryptocurrencies: Casascius physical Bitcoin coins (2011-2013), OpenDime USB sticks, BIP-38 paper wallets, and the various scratch-card "crypto gift cards" sold at convenience stores. These work, technically – a Casascius coin is a genuine bearer instrument – but they are workarounds, not designs. The underlying token (Bitcoin) does not natively understand bearer-paper form. Loss of the physical coin is permanent loss of the value. Counterfeiting requires faith in the manufacturer's hologram, not in the cryptography.³

What if the bearer-paper form were a *first-class* instrument of the system? Designed in from the start, with the same cryptographic and economic properties as on-chain balances, integrated with the identity and reserve mechanics that already exist?

That is what BUCK Notes is.

1.1 Why Forty Years of Trying

David Chaum invented digital cash in 1982 with the blind-signature e-cash scheme.⁴ By 1989 he had founded DigiCash to commercialize it. By 1998 DigiCash was bankrupt – not because the cryptography failed (it worked), but because no merchant wanted to be the first to accept it and no consumer wanted to be the first to hold it. The two-sided market never bootstrapped.

Stefan Brands extended Chaum's work in 1993 with offline divisible coins that revealed the spender's identity if (and only if) they double-spent.⁵ Camenisch, Hohenberger, and Lysyanskaya improved on this in 2005 with compact divisible e-cash.⁶ None of these schemes ever shipped in

¹Federal Reserve. "Currency in Circulation" (2024). Approximately half of US currency in circulation by value is held outside the United States, providing a global store-of-value function despite zero yield.

²Bank for International Settlements. "Rise of the Central Bank Digital Currencies" (2020). Surveys CBDC designs across major central banks; notes that nearly all incorporate explicit issuer-side surveillance and programmability features (spending limits, geofencing, expiration, freezing) that physical cash does not have.

³Caldwell, M. "Casascius Physical Bitcoins" (2011-2013). Physical brass and silver coins containing Bitcoin private keys under tamper-evident holographic seals. Production halted in 2013 after FinCEN classified the issuer as a money transmitter requiring federal licensing. Surviving "unpeeled" Casascius coins now trade at substantial premiums above their Bitcoin face value as collectibles.

⁴Chaum, D. "Blind Signatures for Untraceable Payments" (1982). *Advances in Cryptology - Crypto '82*, pp. 199-203. The foundational paper introducing blind signatures and digital cash. Chaum's DigiCash company commercialized the design from 1989-1998 before bankruptcy.

⁵Brands, S. "Untraceable Off-line Cash in Wallets with Observers" (1993). *Advances in Cryptology - Crypto '93*, pp. 302-318. Offline divisible e-cash with double-spend identity revelation – a deterrent rather than prevention model.

⁶Camenisch, J., Hohenberger, S., Lysyanskaya, A. "Compact E-Cash" (2005). *Advances in Cryptology - Eurocrypt*

production at scale.

Bitcoin, when it arrived in 2009, was deliberately *not* digital cash in the Chaumian sense. It was digital scarcity with public ledger transparency – closer to digital gold than digital cash. The bearer-paper market that grew up around it (Casascius, OpenDime, paper wallets) was an afterthought, not a design goal.

Zerocash (2014)⁷ and its production deployment as Zcash gave us the first scalable on-chain commitment-and-nullifier UTXO model – the cryptographic primitive that BUCK Notes use. Zcash showed that you can have private on-chain balances; it did not extend that to printable physical bearer instruments. Aztec⁸ extended the model with predicate-encrypted notes for private smart contract calls, but again, no paper.

Tornado Cash⁹ reduced the primitive to its simplest form (fixed-denomination commitments in, anonymous withdrawals out) and was OFAC-sanctioned in 2022. Privacy Pools¹⁰ extended Tornado with per-spend association-set proofs, restoring compliance paths without abandoning the privacy primitive.

The reason all of this never produced BUCK Notes – a printable, demurrage-aware, identity-aware private bearer instrument – is that it required four pieces to come together simultaneously, and they did not until now:

1. A **commitment-and-nullifier UTXO model** on a smart contract platform (Zerocash gave us this in 2014; Aztec deployed it on Ethereum in 2018-2020).
2. A **cryptographic identity layer** that lets some notes be targeted to a specific recipient and others be self-wormholed between a user's own unlinkable accounts (the BUCK identity system, with PS signatures, ElGamal credentials, and Chaum-Pedersen re-encryption – see Identity).
3. A **demurrage mechanism** that solves the lost-cash problem and gives bearer notes a finite lifespan (BUCK's demurrage and Jubilee are unique among monetary systems).
4. A **jurisdictional reserve** that backs the value, so the bearer note represents a real claim on real assets, not a speculative crypto-token whose price might collapse between print and deposit (Alberta Buck's commodity-basket-stabilized reserve and the BuckK PID controller).

Alberta Buck has all four. No prior system had all four. That is why BUCK Notes is a *design*, not a workaround.

⁷Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., Virza, M. "Zerocash: Decentralized Anonymous Payments from Bitcoin" (2014). IEEE S&P 2014. The foundational design for on-chain commitment-and-nullifier UTXO privacy, deployed in production as Zcash since 2016 – the commitment-and-nullifier primitive used by BUCK Notes.

⁸Williamson, Z., et al. "Aztec Protocol" (2018-). Ethereum L2 rollup providing general-purpose private smart contracts using PLONK proofs and predicate-encrypted notes. The cryptographic ancestor of the **predicate** field in BUCK Note commitments.

⁹Pertsev, A., Semenov, R., Storm, R. "Tornado Cash Privacy Solution" (2019). Fixed-denomination commitment-and-nullifier mixer on Ethereum. Sanctioned by the US Office of Foreign Assets Control (OFAC) in August 2022 for alleged use in money laundering by North Korean state actors.

¹⁰Buterin, V., Schaefer, J., Tromer, E., et al. "Blockchain Privacy and Regulatory Compliance: Towards a Practical Equilibrium" (2023). Extends commitment-pool mixers with per-spend "association set" proofs, allowing depositors to demonstrate non-membership in incriminating subsets without revealing their specific deposit.

1.2 BUCK Notes at a Glance

Before the chapter-length treatment, here is the entire design in one page.

One on-chain primitive. A single Merkle tree of note commitments and a single nullifier set replace every parallel bearer-instrument architecture the design would otherwise need (self-wormholes between private accounts, signed cheques, targeted gifts, printable paper notes).

Two orthogonal axes parameterise a note. *Depositor binding* (addressed: only the named recipient Identity may deposit, from *any* of its accounts; bearer: any registered Identity holding the opening may) crosses with *issuer visibility* (public: the issuer’s Identity M is in the clear; private: encrypted to the recipient). Crossing these yields three realisable flavours – the private-bearer cell is impossible (mutual decryptability):

- **A1 – Addressed, Public Issuer.** A cheque to a named recipient Identity from a publicly-attributable maker (Schnorr-signed). Payroll, audited disbursement, B2C.
- **B1 – Bearer, Public Issuer.** A cashier’s cheque: maker known, bearer not, cash-on-presentation. At deposit the depositor reveals *its* Identity to the (public) issuer – the depositor-binding dual.
- **A2 – Addressed, Private Issuer.** A sealed private cheque: the issuer encrypts its *own* Identity to the recipient, who alone can read it; the canonical *self-wormhole* between a user’s own unlinkable accounts is the degenerate case.

One naming relation, two gates. Each flavour publishes an ElGamal ciphertext of a registered Identity under a target key. The spend proof is not one circuit three times. A1 and A2 fold, because reading the note and being the named Identity are two secrets. B1 keeps a sigma beside a membership proof, because both of its claims use the depositor’s one scalar, and that membership proof uses a generator with no known discrete log. Each direction is collusion-resistant by membership of the decrypted counterparty, and each party can produce a plaintext receipt naming both. The model is *Mutual Decryptability* below.

Demurrage needs no new machinery. The Notes Pool *is* simply the Notes contract’s own ERC-20 balance – a regular BUCK account tracked by the existing demurrage ledger. Mint pulls `totalFace` BUCK from the caller via standard ERC-20 transfer (sender pays their own outgoing demurrage fee). Spend uses a *carrying transfer* unique to the Notes contract’s single egress path: the recipient’s nominal balance rises by the *full* face value, and a share of the pool’s accumulated `buckSeconds` (its demurrage age) is apportioned onto them, so they inherit the average pool-residence demurrage on the inflowing share. Donations to the contract dilute the pool’s per-BUCK age, directly lowering the demurrage absorbed by all subsequent spends. Conservation is automatic; the Jubilee continues to grow autonomously per BUCK’s standard accounting.

Lost notes return value to the commons. Demurrage solves physical cash’s oldest problem: a lost note’s value decays at the same rate as an on-chain balance, with the difference accruing to the Jubilee rather than as seigniorage to the issuer.

Holding semantics. Addressed notes (A1, A2) are cryptographically safe for long-term holding: only the recipient Identity can spend, and – because authority is the *Identity*, not one account key – a lost account key is *recoverable* by re-registering a fresh account on the same Identity (see *Long-Term Holding Safety*). The *bearer* flavor (B1) is a civil-trust instrument: the issuer necessarily knows ρ and could race the holder, so B1 cheques are meant to be cashed promptly – intrinsic to the bearer predicate, not a policy choice.

The remainder of this document develops each piece in detail and establishes the formal proofs (Theorems 7-11 of the companion *Proofs* document).

2 The Core Primitive: One Global Commitment Pool

The entire BUCK Notes design rests on a single on-chain primitive: a Merkle tree of commitments with a nullifier set, drained by an ordinary BUCK **transfer** that – because the pool is a Carrying account – BUCK routes through its internal *carrying-transfer* path, delivering the opening’s face value to a recipient and apportioning a share of the pool’s accumulated buck-seconds (its demurrage age) onto them. Everything else – paper notes, addressed cheques, bearer cheques, self-wormholes – is a consequence of varying the *spend predicate* inside the commitment.

2.1 The Note Contract State

The Notes contract is a separate, dedicated contract (`src/Notes.sol`) holding a Tornado-style incremental Poseidon-T3 Merkle tree, a nullifier set, and a small ring buffer of historical roots.

State	Type	Purpose
<code>nullifiers</code>	<code>mapping(uint256 => bool)</code>	Double-spend prevention
<code>noteFaceSum</code>	<code>uint256</code>	Total face value of all outstanding notes (audit scalar)
<code>nextLeafIndex</code>	<code>uint32</code>	Next free leaf slot (also = current leaf count)
<code>roots[30]</code>	<code>uint256[ROOT_HISTORY_SIZE]</code>	Recent root ring buffer; spends may use any historical root
<code>currentRootIndex</code>	<code>uint32</code>	Newest entry in <code>roots[]</code>
<code>mintVerifier</code>	<code>IMintVerifier (MintVerifierAdapter)</code>	Batch-mint verifier; the adapter dispatches by <code>cms.length</code> to the per-N Groth16

The tree is fixed at depth $d = 20$ ($\approx 1,048,576$ leaves) and uses Poseidon-T3 (2-input) at every internal node. The per-leaf Merkle insertion logic lives *inside the mint SNARK*; the contract itself does not maintain `zeros[]` or `filledSubtrees[]` and does not expose an on-chain `_insert()` entry point. Every mint submits a SNARK-attested `newRoot` which the contract writes into `roots[]` in one step. The ring buffer holds the 30 most recent roots so spends remain valid for a short window after new notes are inserted, without requiring per-spend root-update synchronisation. The contract keeps no per-commitment `commitmentExists` duplicate-reject map – duplicate mints are economically self-punishing via the deterministic nullifier $\text{Poseidon}_3(\rho, \text{idHash}, 4242)$, so the chain does not need an explicit check.

The *Notes pool* – the BUCK custody side of the design – is just the Notes contract’s own ERC-20 balance. No separate `poolBalance`, `poolAge`, or `poolT` state is maintained: the BUCK ledger’s own per-account balance and demurrage index for `address(notes)` already are the pool, and `noteFaceSum` is a pure audit scalar (the total face-value liability outstanding). The pool is bound in the IdentityRegistry as a Public Identity (`isPublicIdentity`) so identity-bound transfers from any registered party can land at it, and `isCarrying` so its egress on spend takes the Carrying-transfer path.

BUCKs transferred to the Notes contract on mint join its ERC-20 balance through the normal path. BUCKs transferred out on spend use the very same ERC-20 **transfer** – but because the pool is registered `isCarrying` in the IdentityRegistry, BUCK routes the egress through its internal Carrying-transfer path (`Buck._carryingTransfer`), which moves the face value *and* apportions a proportional share of the pool’s accumulated `buckSeconds` (its demurrage age) onto the recipient. No new BUCK entry point is involved – the carrying behaviour is dispatched by the sender’s `isCarrying` flag; see "Demurrage at the Pool Level" below.

2.2 The Note Commitment

A note commitment is a Poseidon hash of the fields the SNARK will later open:

$$cm = H_{cm}(\text{flavor}, v, \rho, \text{id-payload}, \text{predicate})$$

with:

- **flavor** $\in \{A1, A2, B1\}$: the Identity-binding flavor
- **v** $\in \mathbb{Z}_q$: face value
- **ρ** : per-note randomness (source of the nullifier)
- **id-payload**: Identity-binding data (see "Note Flavors" section)
- **predicate**: optional additional spend predicate (time locks, oracle conditions, multi-sig; not elaborated in detail here – a single bit **predicate** == **None** keeps the SNARK to the three base flavors. See "Open Questions" for bounded-programmability notes.)

No per-note timestamp or baked-in BUCK-age: demurrage is computed at redemption time from the *pool's* account state (B, A) , not from the individual note's life history, so no per-note age fields need to ride in the commitment¹¹.

The commitment is inserted as a leaf of **noteTree** at mint time.

2.3 The Note Nullifier

The nullifier is a Poseidon PRF over the note's secret opening material – one derivation for every flavor (the shipped **spend.circom**):

$$nf = \text{Poseidon}(\rho, idHash, 4242)$$

The constant tag domain-separates nullifier preimages from the Poseidon-5 commitment preimages, so a $(\rho, idHash)$ collision between a commitment and a nullifier is structurally impossible.

No spend-side account key enters the hash. Spend *authorization* is layered on top of the nullifier, per flavor: for B1 (bearer), knowledge of the opening *is* the authorization – whoever holds ρ spends, first come first served. For A1/A2 (addressed), the folded deposit gate – one proof, which re-derives this same nullifier in-circuit from $(\rho, idHash)$ as its tie to the spent note – requires the depositing account to be bound to the recipient identity m_{rec} and a certified leaf to pair that identity with the key that reads the note. Because no mint-time key-pair is baked into the nullifier, key loss is not terminal for an addressed note: the recipient binds a *new* account to the same identity and spends – the identity, not the account key, is the credential.

The issuer knows $(\rho, idHash)$ and can pre-compute nf : she can *watch* for her note's redemption (the non-deniable-receipt posture), but cannot *spend* an addressed note – the deposit gate demands a sigma binding the depositing account to m_{rec} .

The nullifier is emitted on deposit; double-spend is rejected by set-membership check.

2.4 Lifecycle

mint (burn, batch):

```
# The caller supplies a batch of N commitments, the SNARK-attested
# newRoot, and the oldRoot / nextLeafIndex they based the proof on.
# The per-leaf Poseidon insertion is inside the SNARK, not on chain.
require cms.length > 0
```

¹¹The astute reader will recognize that this essentially *freezes* demurrage for BUCK Note amounts while they are in transit; this has beneficial effects for the transmission of large long-term BUCK balances, such as inheritances. The cost of this benefit is effectively amortized across all BUCK Note users; the value of Anonymity for short-term Note users subsidizes long-term Note holders, who bear some additional risk that their Notes may be lost]

```

require issuerMode.length == cms.length
require oldRoot == roots[currentRootIndex] # stale-state guard
require nextLeafIndex == self.nextLeafIndex # stale-state guard
# cm[] and issuerMode[] are public inputs DIRECTLY (no batch hash); the
# MintVerifierAdapter dispatches by cms.length to the per-N verifier.
require mintVerifier.verifyMint(
    proof, issuerMode, oldRoot, newRoot, nextLeafIndex, totalFace, cms)
# The SNARK has proved, for some witness
# (flavor_i, v_i, rho_i, idHash_i, predHash_i, pathSiblings_i)_i:
# - cm_i = Poseidon5(flavor_i, v_i, rho_i, idHash_i, predHash_i)
# - v_i in [0, 2^128), totalFace in [0, 2^128) # Num2Bits(128)
# - sum(v_i) == totalFace
# - folding {cm_i} onto oldRoot at nextLeafIndex using pathSiblings_i
#   yields newRoot
# - flavor_i in {A1,A2,B1}; issuerMode_i = (A2 ? PRIVATE : PUBLIC)
# Then: if msg.sender is a registered PUBLIC Identity, require a Schnorr
# over keccak(cms) (mutual decryptability). Private (A2) batches instead
# route to the mint_batch_a2 verifier and field-match each leaf's eIss to
# its re-encryption binding.
buck.transferFrom(caller, address(this), totalFace)
# Adopt the SNARK-attested new root and advance the rolling state.
currentRootIndex = (currentRootIndex + 1) % ROOT_HISTORY_SIZE
roots[currentRootIndex] = newRoot
self.nextLeafIndex += cms.length
noteFaceSum += totalFace
emit Minted(caller, totalFace, nextLeafIndex, cms.length, newRoot)

spend (mint-out):
# caller supplies (proof, root, nullifier, face, recipient)
require root in roots[] # any of the last 30 historical roots
require nullifier not in nullifiers
verify SNARK (spend, B-flavor / bearer shape):
- face in [0, 2^128) and face == v # Num2Bits(128) on both
- flavor == B1 and predicate == 0 # entry-point policy is public
- cm = Poseidon5(flavor, v, rho, idHash, predHash)
- issuanceCommitment == cm # B1-only public mint handle
- Merkle inclusion of cm under root # Switcher + Poseidon(2) per level
- nullifier == Poseidon3(rho, idHash, 4242)
- ghost-bind public signals (face, recipient, chainId) via x*x
require publicIssuerOfCommitment[issuanceCommitment] == issuer
nullifiers[nullifier] = true
noteFaceSum -= face
# An ordinary ERC-20 transfer out of the pool. Because the pool is
# an isCarrying account, BUCK auto-routes it through _carryingTransfer:
# the recipient receives face BUCK and inherits a proportional share
# of the pool's accumulated buckSeconds (its average BUCK-age), scaled
# to the inflowing share of their new total balance.
buck.transfer(recipient, face)

```

Full face value is delivered: the recipient's nominal balance goes up by exactly **face**. The pool's demurrage cost rides along as a proportional share of the pool's live **buckSeconds** integral (**carried** = **liveBs** * **face** / **poolRaw**), merged into the recipient's own **buckSeconds**, in proportion to the inflowing share of their new total balance. The recipient bears that demurrage on their subsequent outgoing transfers through the ordinary BUCK accounting path – the same path any holder travels, but on the inherited buck-age acquired by accepting the note. No Jubilee payment is emitted from inside spend; the Jubilee grows autonomously per BUCK's standard **_accrueJubilee** machinery, independent of how BUCK is distributed between accounts.

3 A Note is a Deferred Approve Handshake

The single most useful way to understand BUCK Notes is that *a note is an **approve** handshake split across time and mediated by the commitment pool*. Once this frame is in place, every structural choice below follows mechanically.

3.1 The Regular On-Chain Transfer

A standard BUCK transfer between two registered identities is a synchronous bilateral exchange:

1. Alice calls `approve(bob, v, E_alice_for_bob, CP_proof)`. She re-encrypts her credential under pk_{bob} and proves (Chaum-Pedersen) that the re-encryption carries the same M_{alice} as her registered credential.
2. Bob calls `approve(alice, 0, E_bob_for_alice, CP_proof)`. Same in the other direction.
3. `transfer(bob, v)` consummates the transfer once both halves are recorded.

All three steps may land in the same block, as long as they're available by `transfer(bob, v)`-time. Each party learns the other's M ; every on-chain observer sees both addresses, the amount, and the timing. The transfer edge `(alice, bob, v, t)` is public and permanent.

3.2 The Note is the Same Handshake, Deferred

A Note reorganises the same three messages:

1. **Mint.** Alice performs *her half* of approve against a pool commitment rather than directly against Bob. Depending on whether the depositor is known at mint time, she either encrypts her credential under the depositor's public key (`E_alice_for_depositor`, identity-bound flavor) or publishes her credential in the clear with a bearer secret ρ (public-issuer bearer flavor). The commitment is appended to `noteTree`; Alice's outgoing BUCKs lands in the pool's balance B .
2. **Deposit.** The eventual depositor submits a SNARK-proved spend predicate that simultaneously:
 - opens the commitment and proves nullifier correctness,
 - completes *their half* of approve by attaching `E_depositor_for_issuer` (the mirror El-Gamal ciphertext) as a deposit-event payload, and
 - authorises an ordinary `buck.transfer(recipient, face)` out of the pool, which – the pool being a Carrying account – BUCK routes through `_carryingTransfer`, apportioning a share of the pool's accumulated buck-seconds (its demurrage age) onto the recipient.
3. **Receipt.** The issuer later scans deposit events for the nullifier of the note she printed and reads `E_depositor_for_issuer` to learn who cashed it.

Both parties end up with the same cryptographic knowledge of each other that **approve** would have produced. The only difference from the observer's perspective is that mint and deposit are two independent transactions, and the commitment pool hides which mint corresponds to which deposit; the demurrage liability transferred with v is the Note pool average at that instant¹¹.

3.3 Mutual Decryptability: The BUCK Identity Invariant

Every BUCK transfer, note or otherwise, preserves a load-bearing Identity property: *both parties must end up able to decrypt each other's M* . In **approve**, this is achieved explicitly by the two re-encryption calls. In a Note, the same property must still hold:

- At *mint* time, the issuer has to publish, alongside the commitment, enough material that the eventual depositor can recover M_{issuer} .
- At *deposit* time, the depositor has to publish enough material that the issuer (reading chain events) can recover $M_{\text{depositor}}$.

This invariant is the single strongest constraint on note structure. It is what rules out *pure-bearer anonymous notes* (neither party learns the other's identity), and what forces the exactly-three-flavor taxonomy below.

The authority axis is the Identity, not the account. A direct transfer *deduces* each party's M from their on-chain account; a Note instead *names* the Identities and treats accounts as interchangeable envelopes that need only *match* the expected M . Naming and reading are different keys. The identity scalar m is what a receipt discloses, and every Fountain account of that Identity shares it, so it cannot also be the key that opens the recipient's notes: one disclosure would read every note, past and future. Addressed notes are keyed to a receiving key $pk_{\text{recv}} = kG$, derived from the wallet seed and registered beside the Identity inside one accumulator leaf $\text{Poseidon}(\tau_R, m, k, \text{salt})$, led by its own v2 field tag like every leaf function. The issuer needs $(M_{\text{rec}}, pk_{\text{recv}})$, learned out of band, and *any* of the recipient's accounts can later redeem, because authority stayed with the Identity. Disclosing m names the person. It does not open their mail, and it does not register accounts or authorize spends: only the PS credential σ does that.

That association is committed TWICE, and the duplication is the whole point: the spend proves it in zero knowledge, where the prover holds the scalars, while a payer must check it BEFORE paying, holding no secret at all. So a second leaf, $\text{Poseidon}(\tau_M, M.x, M.y, pk_{\text{recv}.x}, pk_{\text{recv}.y}, \text{salt}')$, commits the same fact over the two POINTS – the two values a payer has anyway – and checking it is a hash and a path. Under a different salt, so the leaf a holder shows a payer does not locate the leaf its spend proves under. The alternative was handing the payer k , which is handing over the mailbox in both directions in time.

One naming relation, two gate shapes. Both halves of the handshake publish an ElGamal ciphertext of a registered Identity under a target key. That is the relation the three flavours share. The proof that spends the note is not one circuit copied three times. It follows from how many secrets the two claims rest on. An addressed note must show that k opens the ciphertext and that the spender is the Identity m the note named. Those are different secrets, so no shared Fiat-Shamir nonce ties them, and proving them as two proofs that happen to mention one public point lets a thief of k spend under its own Identity. A1 and A2 therefore fold both claims, the leaf, and the path to the posted root into one witness. B1's two claims rest on the depositor's single identity scalar, so its sigma's shared nonce is a real tie, and the membership proof beside it must use a generator whose discrete log is unknown. The accumulator has two levels: each registry keeps private subtrees of salted leaves, an aggregator's tree has their roots for leaves, and only its root rt_{id} is on chain, posted into a ring with its time. Every level hashes alike, so membership is one path of 32 levels (a subtree of 12, the aggregator's 20). The three flavours, with $(K, \text{who proves, when})$ fixed:

- **A2** (private issuer, addressed): the issuer encrypts its *own* M_{issuer} under the recipient's receiving key pk_{recv} at mint; the recipient decrypts with its receiving secret and *alone* produces a receipt naming both parties.

- **A1** (public issuer, addressed): the same, but M_{issuer} is in the clear with a signature – the gadget with the encryption dropped.
- **B1** (public issuer, bearer): the depositor encrypts its *own* $M_{\text{depositor}}$ under the *public* issuer key pk_{iss} at spend, emitted in the deposit event; the issuer decrypts with sk_{iss} and alone names the depositor.

A2 and B1 are duals in *who encrypts whose Identity, and when*. They are not duals in the shape of the proof. A1 drops the issuer ciphertext and keeps the folded gate, because the recipient is still addressed. Membership of the decrypted counterparty Identity is what makes each direction collusion-resistant: a bogus ciphertext decrypts to a non-member and the spend reverts, so a colluding pair cannot construct a spendable note whose counterparty is un-nameable. (The same invariant kills the private-issuer bearer "B2"; see *What This Architecture Rules Out*.) The worked cycles are in the flow document, and *Proofs* Part IV, Theorems 7-12, states and proves these gates.

What is on chain. `spendCoupledA1` and `spendCoupledA2` verify `DepositFoldA1Verifier` / `DepositFoldA2Verifier`, and nothing else: the slot cannot be cleared, and an addressed spend with it unset reverts before the spend SNARK is checked, so there is no configuration in which a weaker gate answers for one. The adapter derives every public input, and it reads the depositor's registered key and credential from the registry. `spendCoupledB1` keeps `verifyDepositorBinding` and composes it with `IdentityMembershipB1Verifier`, whose generator is H_{PEDERSEN} – sound there because B1's two facts rest on one secret. `test/NotesE2E.t.sol` wires all three and spends real proofs along 32-level paths. The addressed calldata is the re-encryption and one proof; the spend events publish no committed point, because no relation reads one. Each spend names the identity root it proved against, and `Notes` accepts any root the registry has retained and posted within seven days – the consumer age that is also the protocol's revocation latency. Roots are posted by the aggregator the registry's root authority appoints, and a receiving leaf is admitted by its registry's certification: `register` refuses a caller-supplied leaf, and must, since a call that accepted any leaf would accept one pairing any Identity with any key. The setups are dev entropy.

The Notes contract itself is bound under a Public Identity after deployment. `Deploy.s.sol` wires the contracts but deliberately leaves this ceremony pending; `BindNotesIdentity.s.sol` consumes the operator's target-bound credential once the registry and Notes addresses are known. Both the issuer and the recipient must complete a one-time CP-bound `approve` for the Notes contract address before their first mint or spend. This establishes the per-pair CP fragments that the BUCK transfer guard requires and that the Notes operator decrypts under subpoena. Outside observers see only opaque hashes of independently-randomised ElGamal ciphertexts — no link between mint and deposit.

3.4 The Non-Deniable-Receipt Invariant

Mutual decryptability is the mechanism; the *non-deniable receipt* is the guarantee it buys, and it is the system's headline property. Every BUCK transfer – direct or through the Notes pool – leaves the recipient able to produce a *receipt*: a plaintext, publicly-checkable statement naming a *registered* counterparty Identity and the value, derivable from data the recipient holds plus public chain state. Three clauses make it load-bearing:

- **Completeness.** For every accepted payment the recipient can compute a receipt that verifies and names the payer's registered Identity (A1/B1: the in-clear M_{iss} plus the issuer Schnorr; A2: decrypt $eIss$ under the receiving secret k).

- **Soundness / non-frameability.** No one can forge a receipt naming an Identity that did *not* author the payment. For B1 the payer is the public Schnorr identity recorded against the commitment, and for A1 the fold forces the opened plaintext to be the recipient's OWN Identity, so neither has a third party for a substitution to name. A2 needs one more relation, because an ElGamal ciphertext does not bind its plaintext to one key: a minter could key $eIss$ so that the mint binding is a true statement about its own Identity while the recipient's key opens the same bytes to a DIFFERENT registered Identity, which an honest recipient would then name. So the note commits the binding's blinded $T = r'pk + \gamma H_{PED}$ in its $idHash$, and the A2 fold opens it against the recipient's own key. Re-aiming the key then needs the difference of two Identities as a known multiple of H_{PED} – a discrete logarithm (*Proofs*, Theorem 12). A receipt repeats the check as $M_{named} = C_i - T + \gamma H_{PED}$, with the γ the delivery carried.
- **No deniability under collusion.** Even a willing payer-recipient pair cannot construct an accepted payment with *no* valid receipt: the deposit gate reverts unless the decrypted counterparty Identity is a registry member, so an un-nameable counterparty is *un-spendable*. For B1 (bearer), `verifyDepositorBinding` ties the depositor's *own* Identity to its account, so it names itself. For A1/A2 (addressed), the note $\leftrightarrow eEnc$ tie is a relation INSIDE the folded gate – the circuit recomputes the nullifier from the $idHash$ it opens – and it forces the ciphertext verified at spend to be keyed to *the spent note's* committed identity material (A2: a re-encryption of the committed $eIss$; A1: keyed to the same M_{rec} the note's $eNote$ was addressed to, with the face pinning the plaintext) – so a colluding pair cannot substitute a nameable $eEnc$ for an un-nameable committed $eIss$, and a stolen opening cannot be redeemed with a self-addressed one (*Proofs*, Theorem 8).

The non-frameability asymmetry is the subtle, important part: a colluding pair can *deny or obscure* the issuer – render a deposited note un-nameable, in which case it simply will not spend – but they can never *substitute* a false issuer. The only reachable outcomes are "names the true minter" and "names no one"; under audit the un-nameable case puts the burden on the (always KYC-identifiable) depositor, not on an innocent. (Formal statement and proofs: *alberta-buck-proofs*, Part IV.)

3.5 What the On-Chain Observer Sees

At mint: a transfer of v BUCKs from Alice's address to the contract address, plus a `Minted(issuer, totalFace, startIndex, count, newRoot)` event. The commitments themselves live in the transaction calldata (`cms[]`); each cm_i is a Poseidon hash, so observers learn nothing about the opening, the depositor, the issuer flavor, or ρ .

At deposit: a standard ERC-20 `Transfer` of `face` BUCKs from the Notes contract address to the depositor's address (the Carrying demurrage apportionment is internal – no distinct event), plus the Notes contract's `SpentCoupledA1` / `SpentCoupledA2` / `SpentCoupledB1` event. Observers learn who the depositor is (standard transfer visibility), but not which mint this corresponds to. A `Transfer` whose sender is the Notes pool is a generic pool-egress signal; it does not link mint to spend.

The two transactions are decoupled in time and unlinked in the chain state. Any mass-harvest analysis that tries to reconstruct the `(issuer, depositor, v, t)` edge must either (a) correlate timing across the whole pool anonymity set, (b) subpoena a party with direct knowledge, or (c) break the SNARK / Poseidon.

3.6 Why This Framing Matters

Almost every downstream design choice is a corollary of "deferred approve handshake":

- *Three note flavors* correspond to three valid ways to complete the two halves of the approve across time (next section).
- *Mutual decryptability* is the invariant that kills a fourth flavor (private issuer, unknown depositor).
- *Selective disclosure for compliance* is the natural consequence of each party still holding cryptographic receipts of its counterparty, exactly as in **approve**.
- *Integration with regular transfers* requires no new semantics at the contract boundary: a note deposit is an ERC-20 transfer from the contract address, and the identity-exchange payload in the event is structurally identical to the receipt fragment **approve** already stores.

4 The Surface of a Note: Chain, Message, Paper

A BUCK Note is, fundamentally, a tuple

`(flavor, v, rho, id_payload, predicate, merkle_path)`

that opens to a commitment $cm = H_{cm}(\text{flavor}, v, \rho, \text{id_payload}, \text{predicate})$ in the on-chain note tree. The tuple can live on any medium capable of storing ~200-500 bytes and being read back faithfully. Three surfaces are supported:

4.1 Chain-Only Notes

The opening is transmitted privately from issuer to depositor (for a self-wormhole A2, both are the same user operating from different accounts) through an out-of-band encrypted channel – typically the wallet’s internal storage, or a direct peer-to-peer message between wallets. The note never leaves the issuer’s control infrastructure. The typical self-wormhole case (an A2 note, the private-issuer flavor): Alice mints an A2 note targeting her own M from a private account and deposits it into a second unlinkable account.

4.2 Message Notes

The opening travels as a digital message: an email, SMS, QR shown on a screen, signal-messenger attachment, NFC handoff. No physical artifact exists, but the recipient can store, forward, or print the opening at their convenience. A1 addressed cheques between parties with reliable digital communication occupy this surface.

4.3 Paper Notes

The opening is printed as a QR code on a slip of paper, a card, a steel plate, or a sticker. For addressed notes (A1, A2) – where the cryptography already prevents a photographing observer from spending – no physical seal is required. For bearer B1 cheques – where whoever reads ρ can spend – the QR must be protected by a tamper-evident seal (hologram, scratch-off panel, or laminated sleeve) so the holder can verify they were the first to expose the secret.

A printed note typically also carries:

- A visible denomination (1, 5, 10, 100, 1,000, 10,000 BUCK) for human convenience. The actual value is what the commitment encodes; the printed face is metadata that a wallet verifies before deposit.
- A human-readable issuer mark, for visual recognition. Cryptographic authenticity comes from the on-chain commitment, not the mark.
- Redundant QRs front and back with Reed-Solomon error correction, plus the secret in base32 digits for manual recovery.

Paper notes are the form most people will think of when they hear "BUCK Note." Physical paper is the surface of last resort: it works offline, without batteries, across borders, through power outages, and past the expiration of any given digital communication channel.

4.4 Counterfeiting is Cryptographically Impossible

A "fake" BUCK Note is any opening whose claimed commitment does not appear in the on-chain note tree. The Merkle inclusion proof inside the spend SNARK fails. No mint occurs. No second-guessing is required: there is no paper-inspection threshold to deceive and no "close enough" forgery that a harried cashier might accept.

This is a stronger anti-counterfeit guarantee than the Bank of Canada provides for physical \$20 bills. Polymer banknotes can be counterfeited (the techniques are well-published; the question is only whether the counterfeit passes casual inspection). A BUCK Note cannot be counterfeited at all – the on-chain commitment registry is the authentic-bill list, and any printed note whose commitment is not in the tree will produce a **FAKE** wallet status on scan and nothing more.

4.5 Wallet-App Status Codes

A user-facing wallet that scans a BUCK Note (from chain storage, message, or paper QR) displays one of five statuses:

Display	Meaning
VALID: 1,000 BUCK	Commitment in tree, nullifier unspent, spend predicate satisfiable
DEAD	Commitment in tree, nullifier already spent
FAKE	Commitment not in tree
EXPIRED	Predicate includes an expiry that has passed
NOT FOR YOU	Addressed (A1/A2) note whose bound M_rec does not match the viewer's Identity

Casual users do not need to understand the underlying cryptography any more than they need to understand the security features of a \$20 bill. They scan, see **VALID**, and accept the note as payment.

On Deposit, the wallet constructs the SNARK locally (or via a delegated prover service for thin clients), submits the transaction, and displays the result. SNARK construction for a single-note deposit is on the order of seconds on a modern smartphone – Aztec's Noir compiler and Zcash's Halo2 prover both run on mobile.

After successful deposit, the note's nullifier is consumed and the QR is permanently dead. A torn-in-half paper note is a *human* signal of finality, not a cryptographic one – but it's a useful ritual to prevent confused future holders from trying to redeposit a dead QR.

5 Note Flavors: Depositor Binding x Issuer Visibility

Every note is characterised by two orthogonal choices:

- **Depositor binding** – *addressed* (A) or *bearer* (B). An addressed note names the recipient Identity M_{rec} at mint and is *keyed* to that Identity’s registered receiving key pk_{recv} , so only that Identity may redeem – from *any* of its Fountain accounts. A bearer note names no recipient, so any registered Identity holding the opening secret ρ may redeem.
- **Issuer visibility** – *public* or *private*. The issuer’s Identity M_{issuer} is either attached in the clear (with a Schnorr signature) or encrypted under the recipient’s receiving key pk_{recv} so only the counterparty recovers it.

Why a receiving key and not the Identity point. The Identity scalar is a *read capability*: the design hands it to every counterparty, because that is how a receipt names a person. So it cannot also be what opens the recipient’s mail – one disclosure would read every note the recipient ever receives, past and future. Worse, encrypting a point under itself leaves $C = m(G + R)$, where message and key share a secret and the Diffie-Hellman hardness protecting every other ciphertext here is simply absent: one scalar multiplication per candidate would identify a recipient from public calldata. Notes are therefore keyed to $pk_{\text{recv}} = kG$, a secret the recipient hands to nobody, and the registry’s accumulator commits the pair $(M_{\text{rec}}, pk_{\text{recv}})$ in one leaf so the spend can prove they belong together. Authority remains the Identity; only the mailbox key is separate.

Crossing the two axes nominally produces four cells, but the *mutual decryptability* invariant of the previous section immediately rules out one of them:

Issuer / Depositor	Depositor M-bound (A)	Depositor M-unbound (B)
Public issuer (1)	A1 – identified cheque to a specific registered recipient (payroll; audited disbursements; B2C with attribution)	B1 – bearer cheque from a known issuer (pre-printed cashier’s cheque; any registered identity deposits)
Private issuer (2)	A2 – private cheque to a specific registered recipient (addressed cheque; self-wormhole is degenerate issuer == recipient)	B2 – impossible : a private issuer with no pre-known depositor has no pk under which to encrypt M_{issuer} for the eventual depositor to recover.

The three valid flavors – **A1**, **A2**, **B1** – are *not* three different cryptographic primitives. They are the *one gadget* of the previous section – prove a re-encryption of a registered Identity under a target key – with the target key and the encrypting party fixed three ways. Membership of the decrypted counterparty Identity in the accumulator rt_{id} makes every direction collusion-resistant. The sketches below give each flavour’s shape; the full mechanics and executable walkthroughs are in *alberta-buck-notes-flow*, and the formal proofs in *alberta-buck-proofs* (Part IV).

Why bearer forces a public issuer (B2 is impossible). A bearer note has no recipient key at mint, so the unknown future depositor can recover the issuer’s Identity *only* if it is published in the clear – *bearer* $\Rightarrow$ *public issuer* is forced by mutual decryptability, not chosen. Any binding a private issuer could make for an *unknown* bearer is openable either by every holder of ρ (no privacy) or by no one (no receipt); so a private issuer must *address* the note (A2). The mint circuit enforces this with a per-leaf **issuerMode** public signal: a bearer leaf provably projects to PUBLIC, and there is no B2 codepoint, so a private-issuer bearer mint is *unprovable*. (Decryptability companion, *Why B2 is impossible*.)

5.1 Flavor A2: Addressed Cheque, Private Issuer

The canonical private-to-private payment (and the *self-wormhole*, the degenerate issuer $\equiv$ recipient case – a user moving value between their own unlinkable accounts). *Layman’s view*: a sealed private

cheque made out to a person, that only that person can cash, and that names its writer only to them.

The issuer encrypts its *own* registered Identity under the recipient's receiving key:

$$eIss = (R_e, C_e) = (r'G, M_{\text{issuer}} + r'pk_{\text{recv}}),$$

decryptable by the receiving secret k – so the issuer needs only $(M_{\text{rec}}, pk_{\text{recv}})$, learned out of band, and *any* recipient account can redeem, because redemption authority is the Identity and the accumulator leaf pairs the two. A blinded re-encryption proof (`verifyIssuerReenc`) binds $eIss$ to the issuer's *own* registered Identity under a key it blinds, and publishes $T = r'pk_{\text{recv}} + \gamma H_{\text{PED}}$, which the note commits: $idHash = \text{Poseidon}_{10}(eNote, eIss, T)$. At deposit the recipient proves the folded gate (`deposit_fold_a2.circum`): k opens the ciphertext, the account credential opens to M_{rec} , a leaf commits (m_{rec}, k) under the recipient's salt, a second leaf commits the decrypted issuer Identity under the salt the issuer shipped, and T opens against k with the γ the delivery carried. A bogus plaintext is not a member of that second leaf, so the spend reverts; a plaintext re-aimed at a registered puppet fails the tie, so the recovered issuer is the true minter, never a framed victim. From k and its own Identity the recipient produces a plaintext receipt naming both parties. The worked cycle is in the flow document, "A2 in Code".

5.2 Flavor A1: Addressed Cheque, Public Issuer

The same addressed shape as A2, but the issuer's M_{issuer} is in the clear with a Schnorr signature – *the gadget with the encryption dropped*. The recipient reads the issuer directly (no decryption); the issuer chose the recipient at mint, so it already knows them. Eligibility and redemption are A2's: any account bound to M_{rec} may deposit, gated by the same membership check over the leaf that pairs the Identity with its receiving key. Public issuers are institutional (employers, charities, payroll processors) whose Identity is already public, so naming it in the clear adds audit value at no privacy cost to the recipient (whose account and amount are the only on-chain signals, mitigated as usual).

Use cases. Payroll from a named employer to an identified employee, charitable disbursement to an identified grantee, attribution-preserving B2C payment where the customer wants a cryptographic record that a specific merchant paid them.

5.3 Flavor B1: Bearer Cheque, Public Issuer

The classic cashier's cheque: the issuer is public, the depositor is unknown at mint, and whoever first presents ρ plus a registered credential redeems. The depositor recovers the issuer trivially (it is in the clear with a signature).

The reverse direction – the issuer learning *who* cashed the note – is the **dual of A2**. At deposit the depositor encrypts its *own* Identity under the public issuer key,

$$E_{\text{dep} \rightarrow \text{iss}} = (rG, M_{\text{depositor}} + rpk_{\text{iss}}),$$

emits it in the deposit event, and proves (`verifyDepositorBinding`) that it encrypts a registered Identity bound to its payout account – revealing nothing to Mallory (the ciphertext is IND-CPA under pk_{iss}). The issuer scans the event, decrypts with sk_{iss} , and *alone* produces a receipt naming the depositor (`make_issuer_receipt`). The binding is collusion-resistant: the shared witness couples "account bound to $M_{\text{depositor}}$ " and "ciphertext encrypts $M_{\text{depositor}}$ ", so a depositor cannot cash a B1 note and stay un-nameable. Full construction + tests: the notes-identity-axis unification (see "Mutual Decryptability" and the one-gadget definition below).

Civil-trust caveat (not cryptographic). The issuer knows ρ and can race the bearer to deposit – but any such deposit names the issuer on chain, so an issuer-turned-thief is identified at the moment of theft. Same trust model as a paper cashier’s cheque: cash promptly; trust the issuer’s legal standing and operational discipline. This is intrinsic to the bearer predicate, not a flaw.

5.4 Comparison of the Three Flavors

Property	A2 private issuer	A1 public issuer	B1 bearer, public issuer
Primary use	Private remittances; self-wormhole	Payroll; charity; B2C	Cashier’s cheques; pre-printed bearer
Target key K of the gadget	recipient receiving key pk_{recv}	– (issuer in the clear)	public issuer key pk_{iss}
Who encrypts, when	issuer, at mint	– (plaintext + sig)	depositor, at spend
Recipient names issuer via	decrypt $eIss$ under k	plaintext M_{iss}	plaintext M_{iss}
Issuer names depositor via	chose the recipient at mint	chose the recipient at mint	decrypt event under sk_{iss}
Who can redeem	any account bound to M_{rec}	any account bound to M_{rec}	any registered holder of ρ
Receipt producible by	recipient <i>alone</i>	either party	issuer <i>alone</i>
Collusion-resistant naming	yes (membership)	yes	yes (membership)
Civil-trust caveat	none	none	issuer knows ρ (cash promptly)

(*Addressed spend.*) Given a ciphertext E keyed to $pk_{\text{recv}} = kG$ and a depositor account, prove knowledge of $(m, k, sk, salt)$ such that k opens E to the point the flavour names, the account credential opens under sk to $M = mG$, and $\text{Poseidon}(m, k, salt)$ lies on a path to the posted root. A2 adds that the opened point is itself a registered Identity under the issuer’s shipped salt. One proof; no shared public point.

(*Bearer spend.*) Given a ciphertext E keyed to the issuer’s registered account key pk_{iss} , prove knowledge of one scalar m_{dep} such that the payout account is bound to $M = m_{\text{dep}}G$, E encrypts that same M under pk_{iss} , and $P = M + bH_{\text{PEDERSEN}}$ opens a salted membership proof. The shared nonce is a tie because both facts use m_{dep} . $\log_G H_{\text{PEDERSEN}}$ is unknown.

The taxonomy is those two statements, chosen by which party is private and which party is proving. A fourth flavour – bearer with no identity hook on either side – is intentionally absent (see *What This Architecture Rules Out*).

6 The Interaction Matrix: Note Flavor x Depositor Account Mode

The Identity system (Identity) gives registered accounts two visibility modes:

- **Anonymous-valid.** KYC-verified but Identity encrypted. A counterparty learns M only by completing the bilateral **approve** handshake (or being targeted by a note whose payload pre-completes that handshake).
- **Public.** M is published in the clear. No handshake is needed to know who the account belongs to.

Permissionless (unregistered) accounts cannot appear on either side of a BUCK transfer – regular or Note – because the mutual-decryptability invariant requires a registered identity at every endpoint.

Crossing the three valid flavors with the two depositor account modes produces the six operational cells below. Rows are note flavors (A1 / A2 / B1); columns are the depositor’s account visibility at deposit time. The *issuer’s* account mode is fixed by the flavor itself: A2 issuers are anonymous-valid (or the bearer-issuing contract acting as a pseudo-identity), A1 and B1 issuers are public.

Flavor	Anonymous-valid depositor	Public depositor
A2 (private cheque)	Private-to-private addressed cheque Issuer (anon-valid) targets a specific anon-valid recipient. No on-chain link between them; both learn the other’s M at spend. Self-directed <i>wormhole</i> is the degenerate subcase (issuer = recipient). Use: remittance, inheritance, B2B payments.	Private-to-public addressed cheque Issuer (anon-valid) targets a public recipient. Recipient deposits to their public account; observers see only that a public identity received a pool note. Use: moving private wealth to a public persona; donating to a public charity without a traceable direct transfer.
A1 (identified cheque)	Public-to-private identified cheque Public issuer targets an anon-valid recipient. On-chain observers see the issuer’s M plus a deposit from some anon-valid account; they cannot link the account to its real Identity. Use: payroll, B2C payments, audit-friendly disbursements from a public treasury.	Public-to-public identified cheque Both parties public; the note adds /timing indirection/ over a direct transfer : the cheque can be pre-signed and held. The issuer is already on-chain; the recipient may choose among their public accounts at deposit time. Use: pre-signed instruments, escrow cash-outs.
B1 (bearer cheque)	Bearer-to-private Public issuer prints an unaddressed cheque; first registered spender to publish ρ wins. Civil-trust instrument: theft is not cryptographically prevented, only bilaterally traceable. Use: cashier’s cheques, physical coupons, prize vouchers, out-of-band bearer drafts.	Bearer-to-public Same, but depositor is public. Observers see a public identity cashed a B1 draft from issuer M . Use: redeeming a bearer instrument received out-of-band into a publicly attributable account.

Observations:

1. **Wormhole functionality** – the private same-identity round-trip is the degenerate subcase of A2 where issuer and depositor share the same M . It uses the same mint construction, the same spend circuit, and the same pool; no separate instrument is required.
2. **Cashier’s cheques are B1** and are the only flavor where the recipient identity is *not* determined at mint time. All other flavors bind a specific recipient by addressing the recipient Identity point M_{rec} . This is also the only flavor exposed to issuer race-condition risk – whoever publishes the nullifier ρ first takes the value.

3. **Public-to-public A1 cells** are degenerate in information content: a direct **transfer** carries the same on-chain signal. Their utility is operational (pre-signing, mailing, delayed destination choice), not privacy-structural.
4. **Issuer and depositor account modes are chosen independently**, subject only to the flavor's constraint on the *issuer* side ($A2 = \text{private issuer}$; $A1/B1 = \text{public issuer}$). The depositor's mode affects only what the on-chain observer sees at deposit, never what the counterparties see of each other.
5. **The B2 cell (private issuer + bearer) does not exist.** A private issuer must encrypt under a specific recipient's Identity point M_{rec} so both parties can decrypt each other's M ; there is no recipient to key against in a bearer construction. Attempting a "private bearer" instrument violates the BUCK mutual-decryptability invariant.

7 Integration with On-Chain Transfers

Notes are not a separate payment system. They are an alternative framing of a BUCK transfer in which the **approve** handshake is mediated asynchronously through the pool instead of being settled synchronously between two accounts. Everything that is true of a regular **transfer** – demurrage accounting, Jubilee conservation, sanctioned-account blocks, fee and gas mechanics – is also true of a Note mint or deposit, because both sides of a Note’s lifecycle *are* regular ERC-20 transfers against the pool’s own account.

7.1 The Pool is a Regular Account

The Notes contract’s own address is a registered account in the ordinary sense (system-public in IdentityRegistry). BUCKs flowing in (mints, donations) is a standard incoming transfer; BUCKs flowing out (spends) are standard transfers BUCK routes through its Carrying path (the pool is **isCarrying**). The only contract-local state not already tracked by the BUCK ledger is the Merkle-tree root history (**roots[]**, **nextLeafIndex**), the nullifier set (**nullifiers**), and **noteFaceSum** (aggregate live face value – an audit scalar, not a liability figure).

7.2 Regular Transfer vs. A-Flavor Note

A regular **transfer(alice, bob, v)** followed (or preceded) by **approve(alice, bob)** has the following on-chain signature:

- Alice’s account: **face** debited from her balance, plus her accumulated fee burned and her **buckSeconds** re-crystallised from the new balance.
- Bob’s account: balance credited by **face**; the inflow begins accruing age from now – clean BUCK, no inherited fee.
- Edge observable: Alice $\rightarrow$ Bob, amount **face**, block t .

The A-flavor Note equivalent (A2 between two anon-valid accounts):

- Alice mints: standard ERC-20 transfer of **totalFace** to the Notes contract. Alice pays her own outgoing-transfer fee and re-crystallises her **buckSeconds**, exactly as in a regular transfer. The pool’s balance gains **totalFace**.
- Bob spends at time $t + \Delta$: the pool’s Carrying transfer delivers **face** to Bob and apportions a share of the pool’s accumulated **buckSeconds** onto Bob. Bob now bears the pool’s averaged demurrage on the inflowing share of his balance.
- Edge observable: Alice $\rightarrow$ pool at t ; pool $\rightarrow$ Bob at $t + \Delta$. The edge connecting them is erased.

Economic outcomes match a regular transfer up to the timing difference: Bob acquires the pool’s averaged BUCK-age on the inflow, regardless of how long Bob’s specific commitment sat in the tree. This is the key property that makes indefinite holding of A-flavor Notes economically equivalent to holding BUCK – same demurrage rate (averaged over the pool), just with a different edge visible on chain.

7.3 Regular Transfer vs. B-Flavor Note

A B1 cashier’s cheque replaces the bilateral **approve** with a public-issuer signature binding the note to a specific M_{issuer} and leaves the depositor unspecified:

- Issuer mints: **transfer** from issuer to pool. The issuer’s m_{issuer} is bound into the commitment via the Schnorr signature.
- First registered holder to publish ρ deposits: pool $\rightarrow$ depositor at some later time.
- The depositor and issuer end up mutually identified (per the BUCK invariant) through the note’s commitment payload plus the on-chain deposit actor; no third party sees the edge.

A B1 Note is the natural digital analogue of a cashier’s cheque drawn on a bank: the bank (public issuer) has pre-committed the funds; whoever presents the instrument redeems them.

7.4 When to Use Which

Scenario	Use	Why
Two parties already mutually approved	transfer	Simpler, one on-chain call, no SNARK cost
Payer wants no on-chain edge to recipient	A2	Breaks bulk-harvest trace; same demurrage
Public issuer paying a specific recipient	A1	Auditable issuance, recipient visibility hidden
Pre-signed instrument, recipient undecided	B1	Bearer; cash on presentation
Self-wormhole across own accounts	A2 (issuer = recipient)	Breaks the edge between own addresses
Payroll to many private employees	A1 per employee, or A2	A1 when attribution required, A2 otherwise
One-off payment under surveillance concern	A2	Cheapest way to erase the direct edge

7.5 Cost and UX Trade-Offs

Relative to a regular **transfer**, every Note costs additional gas for the SNARK verifier (mint or spend) and requires wallet software capable of producing the proof. For routine payments between counterparties who do not need to defeat chain analytics, a regular **transfer** is strictly cheaper and simpler. Notes are the correct tool when the privacy property – no on-chain edge between payer and recipient – is worth the additional gas and proving time, or when the instrument semantics (deferred recipient choice, pre-signed cheque, paper distribution) are themselves the point.

7.6 Why A2 Costs More: Anonymity Is the Cost

A direct private-to-private **transfer** already achieves the mutual-decryptability invariant – *without* a SNARK and with collusion-resistance for free – because the bilateral **approve** reads *both* parties’ registered records in one call: both Identity anchors are co-present and named at one moment. Splitting that transfer into mint $\rightarrow$ spend through an A2 note is what makes it expensive, and it is worth being explicit about *why*.

- **The two anchors are never co-present.** The issuer’s registered record exists only at *mint* (issuer = `msg.sender`); the recipient’s only at *spend* (recipient = `msg.sender`). No single on-chain call can read both the way **approve** does, so A2 must *blind* the recipient key at mint and re-unite the anchors in the folded deposit gate. That bridging – the mint SNARK, the blinding, the key tie – *is* the entire extra cost, and it buys exactly one thing: mint $\leftrightarrow$ spend unlinkability. There is no free lunch: cheap collusion-resistant mutual decryptability *requires* naming both parties at once, which anonymity forbids.

- **A2 is asymmetric.** The recipient-names-issuer direction is collusion-resistant and self-provable on chain; the issuer-names-recipient direction rests on the recipient's own registered address being `msg.sender` at spend plus the KYC backstop – not a crypto receipt. A B1 (public issuer) note is the *cheap* path: its depositor binding is literally the **approve** handshake, collusion-resistant because the issuer is named. So A2's hardness is specifically *issuer anonymity*.
- **The recipient bears the burden.** An honest recipient verifies an A2 note *at delivery* – decrypt *eIss*, check it yields the issuer's *registered* Identity – before accepting it; a bogus note fails this check and is rejected. So a recipient who deposits a note that later proves un-nameable was negligent or colluding: the suspicion falls on the (always KYC-identifiable) recipient, never on the anonymous issuer.

Shipping consequence. Public-issuer notes (A1, B1) skip the mint-side apparatus; A2 is the premium option whose extra mint binding ($\sim 100K$ gas) is justified only when the payer needs to hide *its own* Identity behind the mint $\rightarrow$ spend split. At spend, *both* addressed flavours pay a $\sim 370K$ -gas note-binding Groth16 (A2's re-encryption tie or A1's addressed tie) – that is the price of "only M_{rec} can spend", not of issuer anonymity; bearer B1 skips it (see *On-Chain Gas*). (Full treatment in flow "Privacy Audit" (EOA Transfer Mirror).)

7.7 Composability

Notes compose transparently with the rest of BUCK:

- *Identity Fountain re-issuance* applies to Note mints and deposits identically to regular transfers. **A-flavor notes are recoverable under re-issuance:** redemption authority is the Identity M_{rec} , shared by *every* Fountain account of that Identity – so a lost account key is recovered by re-registering a fresh account on the same M_{rec} and depositing (see *Long-Term Holding Safety: Identity-Pinning and Recovery*). Only an explicit account-pinned opt-in treats key loss as terminal.
- *Sanctions* and *Jubilee* operate at the account level and reach the pool through its own account state, requiring no Notes-specific logic.
- *Demurrage* is accounted on each B_X, A_X pair including the pool's; pool-rate demurrage is a consequence of this accounting, not a parallel mechanism.
- *Wallet UX* can expose Notes as an alternative **send** mode in the same UI flow that calls **transfer**; the difference between "direct send" and "private send" becomes a single toggle.

8 Demurrage at the Pool Level

The central simplification of this design: **the Note Pool is just a fee "Carrying" BUCK account.** BUCKs transferred to the contract's address join the pool; BUCKs transferred out (as note redemptions) leave the pool. The contract account accumulates BUCK-age like every other account and participates in the ordinary demurrage accounting with no special-case code. The only piece of additional state is a single scalar F , the sum of face values of live notes.

8.1 Reminder: How BUCK Demurrage Works

BUCK demurrage is a balance-time integral. Each account stores, in one packed slot, its `buckSeconds` – the crystallised integral $\int B_X dt$ up to its last touch – plus the `timestamp` of that touch. There is no global `cumIndex` and no per-account `_indexAtLastTouch`; the integral itself is the state.

The fee owed by account X at any moment is

$$\text{feeOwing}(X) = \text{BASE_RATE_PER_SEC} \cdot \frac{\text{buckSeconds}_X + B_X \cdot \text{elapsed}}{\text{SCALE}},$$

i.e. the rate times the live buck-seconds integral: the crystallised part plus the current rectangle $B_X \cdot \text{elapsed}$ since the last touch. This is the κA_X formulation made concrete – BUCK-age A_X is the buck-seconds integral.

On a *standard* ERC-20 transfer the sender's accumulated fee is first *burned* (its outgoing balance is reduced and `totalSupply` drops) and its `buckSeconds` is re-crystallised from the new balance; the recipient's inflowing BUCK begins accruing age from zero and carries no inherited fee.

Once per touch, the contract also "accrues the Jubilee": the system's demurrage for the elapsed interval (`totalSupply` times the rate-slice) is credited to the Jubilee account's balance via a direct slot write (`totalSupply` itself is not mutated by demurrage). This keeps the system-wide invariant

$$\sum_{\text{all accounts } X} B_X = \text{totalSupply (incl. Jubilee)}$$

automatically preserved across every transfer.

8.2 The Pool Account's Demurrage

The Notes pool is an *ordinary* BUCK account (the Notes contract's own ERC-20 balance). Its demurrage state is therefore exactly the packed per-account state the BUCK ledger keeps for everyone:

- $B = \text{balanceOf}(\text{notes})$ – BUCK transferred in via mint and donations, minus BUCK transferred out via spend.
- `buckSecondspool` – the crystallised $\int B dt$ integral, alongside the `timestamp` of the last touch.

The pool's effective fee-owed at this moment is $\text{rate} \cdot (\text{buckSeconds}_{\text{pool}} + B \cdot \text{elapsed}) / \text{SCALE}$. The Notes contract never reads or writes either value: it lives inside the BUCK ledger just like every other account. Spends drain it through the carrying path described next.

8.3 The Age-Preserving Redemption

When a note of face v is spent, the Notes contract calls an ordinary `buck.transfer(recipient, v)`. Because the pool is an `isCarrying` account, BUCK routes it through `_carryingTransfer`, which – unlike a transfer *from* a non-Carrying account – does not hand the recipient age-free BUCK: it apportions a proportional slice of the pool’s live buck-seconds integral onto the recipient (the full face value still lands):

```
liveBs    = pool.buckSeconds + pool.raw * elapsed    // pool's live integral
carried   = liveBs * v / pool.raw                  // proportional share
pool.buckSeconds    = liveBs - carried              // pool sheds its share
pool.balance        -= v
recipient.buckSeconds += carried                    // recipient inherits the age
recipient.balance    += v
```

The pool’s *per-BUCK* age is preserved – its integral and its balance drop by the same proportion – so successive spends from the pool inherit the same per-BUCK age share as long as the pool keeps minting and spending without an intervening standard transfer.

The recipient’s combined buck-seconds after the merge correspond to a weighted average of the two BUCK-ages: the recipient’s prior holding keeps its accrued age, and the freshly received v BUCK carries the pool’s age. When the recipient next makes a standard outgoing transfer they pay the (combined) fee out of their balance, exactly as if they had personally held v BUCK across the pool’s residence period.

This is the cost of anonymity: the recipient inherits the average pool BUCK-age, which is a net *benefit* for any note that has been held longer than the pool’s average age, and a net cost for notes spent quickly into the pool. In practice, fast-circulating notes subsidise slow ones.

8.4 Donations Integrate Smoothly

Any BUCK transferred to the Notes contract *without* minting a corresponding note arrives via a standard transfer that credits the pool fresh, low-age BUCK – diluting the pool’s per-BUCK age (its buck-seconds integral grows slower than its balance). The pool’s per-BUCK fee owed drops; subsequent carrying-path spends therefore deliver recipients a smaller inherited fee-share per face unit. Over a sequence of spends, the donation is consumed as a demurrage subsidy for live noteholders.

Four origin cases, all handled identically:

1. **Accidental transfers.** A user fat-fingers the Notes contract as a recipient. Their BUCK is not burned; it dilutes the pool’s per-BUCK age, and future redeemers inherit marginally less fee.
2. **Philanthropic donations.** A user deliberately sends BUCK to the pool to subsidise noteholders. The subsidy spreads across all future redemptions until the pool’s per-BUCK age rebuilds.
3. **Recovered collateral.** Mechanism-level refunds (e.g., an issuer’s collateral unlock) sent to the pool address.
4. **Rounding / dust.** Integer-arithmetic residuals. Self-healing.

After all notes are eventually spent, any remaining donation principal sits at `noteFaceSum = 0`, `balanceOf(notes) > 0` – unclaimable by spend, but available to subsidise a fresh round of mints if the pool is used again, or recoverable by governance sweep (per Notes contract policy).

8.5 Conservation is Automatic, and Tight

The carrying transfer conserves the pool's and recipient's nominal balances exactly:

- $\Delta B_{\text{pool}} + \Delta B_{\text{recipient}} = 0$.

No fee is burned and `totalSupply` is unchanged on the carry itself; the buck-seconds apportionment is pure bookkeeping. The next time the recipient makes a standard transfer, the inherited fee-share is paid out of their balance and into the Jubilee via the ordinary BUCK accounting path.

Globally, the system-wide invariant $\sum_X B_X = \text{totalSupply}$ (incl. Jubilee) holds across every transfer (carrying or standard) by construction. The Jubilee's required-balance growth is independent of how BUCK is distributed between accounts; it depends only on the elapsed time and `totalSupply`. No pool-specific conservation theorem is required.

8.6 Numeric Example

Suppose the pool holds $B = 10\,000\,000$ BUCK and was last touched 0.5 yr ago, so the pool's accumulated buck-seconds correspond to a 1% fee owed per BUCK (the BUCK base demurrage rate is 2% per year). The pool currently owes the Jubilee 100 000 BUCK against its 10 000 000 face.

Alice spends a note of face $v = 1,000$:

- Alice's BUCK balance rises by exactly 1,000.
- Alice inherits a proportional slice of the pool's buck-seconds: if she previously held 0 BUCK, the full 1%-per-BUCK age attaches to her 1,000; if she previously held 9,000 BUCK of fresh (zero-age) BUCK, the inherited age averages down to $\sim 0.1\%$ of fee per BUCK across her now-10,000 balance.
- When Alice next makes a standard outgoing transfer of v , her accumulated fee on the proportional share is burned out of her balance and credited to `totalSupply` such that the Jubilee can mint into balance. Net: she receives 1,000 face value, pays up to $1\% \times 1,000 = 10$ BUCK as demurrage when she next spends – same economic outcome as if she had personally held the BUCK during the pool's residency.

Now suppose an anonymous donor sends 500,000 BUCK to the pool via a standard transfer. The fresh, zero-age BUCK dilutes the pool's per-BUCK age; the fee owed per BUCK drops. Alice's next spend now inherits a *smaller* per-BUCK age and therefore pays less demurrage on her next outgoing transfer – a direct subsidy from the donor.

The Jubilee's growth schedule is unchanged: it keeps accruing relative to `rate \cdot totalSupply \cdot elapsed` regardless of how BUCK sloshes between accounts.

8.7 Who Bears the Demurrage, Really

The answer is sharply localized: **the Note recipient inherits the pool's accumulated demurrage share on the face value they received and pays it on their next standard outgoing transfer.** The recipient has accepted the note, and with it the demurrage cost of the BUCK sitting in the pool while the note was outstanding.

This is the "price" the recipient pays for accepting a BUCK Note instead of a direct BUCK transfer: they inherit the average pool-residence demurrage, weighted across their prior holding. For notes held longer than the pool's average residence, this is *cheaper* than holding the BUCK

directly; for notes spent quickly into the pool, it is more expensive. The trade-off is explicit in the resulting **buckSeconds** on the recipient's account.

Noteholders (while holding) bear no demurrage: they hold a cryptographic commitment, not a BUCK balance, and the BUCK they implicitly own sits in the pool where its accumulated fee is borne collectively until spent. The issuer (at mint time) pays their own outgoing-transfer fee through the standard ERC-20 path to deposit into the pool. Third parties bear nothing. The Jubilee keeps receiving the BUCK-wide demurrage stream regardless of how BUCK sloshes between accounts.

9 Demurrage Solves the Lost-Cash Problem

This section is about a property that is genuinely novel, with implications well beyond the BUCK system.

9.1 The Hidden Inflation of Lost Cash

Every monetary system that uses physical bearer instruments suffers a constant slow loss to *unrecoverable currency*. Banknotes are lost in fires, dropped down storm drains, forgotten in the pockets of jeans that get washed and shredded, buried in landfills, sealed in walls, hidden by the elderly and forgotten by their heirs.

The Federal Reserve estimates that 5-10% of US paper currency is permanently unrecoverable at any given moment.¹² At a US M0 of approximately \$2.3 trillion, this represents *\$115-230 billion in permanently extracted value* – seigniorage to the Federal Reserve, since the lost notes never get redeemed but the backing assets remain on the Fed’s books.

This is not theft, but it is not neutral either. Lost cash is a regressive transfer: the people most likely to lose cash are the people who use cash most (the elderly, the unbanked, low-income workers paid in cash). The beneficiary is whoever issued the cash, generally the central bank, ultimately the government.

9.2 What BUCK Demurrage Does For Lost Notes

A BUCK Note that is lost in a fire, dropped down a storm drain, or hidden in a wall by a paranoid grandparent is *not* permanently extracted from the system. The face value sits in the pool’s ERC-20 balance, with the pool’s accumulated buck-seconds growing (its per-BUCK age rising) as the pool sits untouched. No spend ever happens.

Two effects balance:

1. The BUCK backing the lost note stays in the pool indefinitely. It does not vanish.
2. Other live-note spenders, through the Carrying transfer, inherit a share of the pool’s accumulated age at each spend. Because the lost note’s backing weighs on `balanceOf(notes)` without a corresponding spend to drain it, the lost-note’s implicit demurrage is paid installment-style by subsequent spenders on their next outgoing transfer.

Over decades, the buck-seconds accruing on the lost-note backing accumulate the demurrage that would have been owed had the BUCK been held in a regular account. The autonomous Jubilee growth continues regardless. Net effect: *the 5-10% of currency that physical cash systems lose forever, BUCK Notes returns to the commons within a generation*, via the steady demurrage pressure that the lost face value applies to live spenders inheriting the pool’s accumulated age.

This has cascading consequences:

1. **Money supply self-correction.** No system-wide accounting tail of "missing" currency that the central authority must track and write off. Lost notes write themselves off through the pool-age mechanism.

¹²Federal Reserve Bank of San Francisco. "Diary of Consumer Payment Choice" (2018). Estimates 5-10% of US paper currency by value is permanently unrecoverable. At a US M0 of \$2.3 trillion, this represents \$115-230 billion in permanently extracted seigniorage.

2. **Anti-hoarding incentive matches on-chain balances.** A BUCK held in your wallet or in your pocket as paper decays at the same rate (via the age-share that will eventually be absorbed on redemption). There is no advantage to converting on-chain BUCK to paper to evade demurrage; the demurrage clock runs the same way in both forms.
3. **Inheritance is automatic.** A grandparent who hides 100,000 BUCK in paper notes and forgets to tell anyone does not extract 100,000 BUCK from the economy on their death. The value flows back through the Jubilee to BUCK issuers – broadly, back into the same economy where the grandparent originally earned it.
4. **No central bank profit from currency loss.** The seigniorage that accrues to central banks today from unrecoverable cash does not exist in BUCK Notes. Lost value does not benefit the issuer; it benefits the system as a whole.
5. **Counterfeit recovery is automatic.** Even if a note printed years ago were somehow forged (which is cryptographically impossible, but humour the counterfactual), the value at stake is bounded by the demurrage decay. An attacker forging old notes would be forging paper whose backing BUCK has already been largely absorbed by live redemptions.

This is, I believe, the first monetary system in history where lost cash returns to the commons by automatic mechanism rather than by accounting fiat. Physical cash cannot have this property, because physical cash has no clock. It emerges naturally from BUCK’s existing demurrage mechanism applied to the note-pool primitive.

9.3 The Symmetry With On-Chain Balances

A second elegance worth pointing out: the effective demurrage rate on a BUCK Note – as absorbed by the recipient at deposit – is *the same* as the demurrage rate on an on-chain BUCK balance that lived in the pool for the same duration. This is by design, and it matters.

If paper BUCK Notes demurraged faster than on-chain BUCK, holders would prefer on-chain (and paper would be discouraged, defeating the design’s purpose). If on-chain BUCK demurraged faster than paper notes, holders would race to print all their on-chain balances as physical notes (and the chain would lose its primary use as a settlement layer).

The matching demurrage rate makes the choice between on-chain and paper purely about *use case fit*: on-chain for routine commerce, automated payments, large transactions; paper for offline payments, gifts, savings under the mattress, disaster preparedness, and so on. Neither form is privileged by the demurrage mechanism.

10 Long-Term Holding Safety: Addressed vs. Bearer

Pool-rate demurrage makes every flavor equally cheap to *hold*. What distinguishes them for long-term storage is whether the spend predicate binds a specific recipient Identity M_{rec} or accepts any registered depositor. Addressed notes (A1, A2) are cryptographically safe for indefinite holding; bearer notes (B1) are civil-trust instruments and should be cashed promptly.

10.1 What the Issuer Knows

At mint time the issuer holds the full commitment preimage $(v, \rho, \text{id-payload}, \text{predicate})$ including, for addressed notes, any randomness r' it used to encrypt the issuer payload under the recipient's receiving key pk_{recv} . The question is whether that knowledge, by itself or combined with any later-acquired secret, lets anyone other than the intended recipient spend the note.

10.2 A-Flavors (Addressed): Cryptographically Safe, and Recoverable

Both A1 and A2 address a specific recipient *Identity* M_{rec} at mint and key the note to that Identity's registered receiving key. The spend gate is *one* proof over *one* witness, carrying four relations:

1. the receiving secret k decrypts the note's ciphertext to the point the gate commits;
2. the depositor's registered account credential decrypts, under its own account key, to the Identity M_{rec} ;
3. a registered accumulator leaf commits the pair (m_{rec}, k) under the holder's own salt;
4. that leaf's path folds to a posted identity root.

A2 carries a fifth, and the reason is the one difference between the flavours: its ciphertext decrypts to the *issuer's* Identity, a point the spender holds no scalar for. So it must also show that decrypted point is itself registered, or a colluding issuer keys the note to a throwaway and leaves the recipient holding garbage no receipt can name. A1 needs no such relation – its plaintext is the recipient's own Identity, which relation (3) already commits. And A2 carries a sixth: the mint binding's blinded T , committed in the note's *idHash*, opens as $(r'k)G + \gamma H_{\text{PED}}$ for the recipient's own k . Registered is not the same as right; relation (5) says the decrypted Identity is registered, and relation (6) says it is the minter's.

The leaf commits the two SCALARS, not the two points. Its siblings commit coordinates because the authorities that compute them hold identity points and nothing else; this leaf exists to be proven in zero knowledge by a holder that has the scalars, and BN254's G_1 group order equals the Poseidon field, so a scalar is a field element outright. Committing points would oblige the circuit to re-derive both from those same scalars at 471,896 constraints apiece – 22% of the gate, to reach a commitment whose preimages the prover already holds. It also binds more tightly: the circuit hashes the very signals its other relations consume, so the Identity in the credential relation and the Identity in the leaf are the *same signal* rather than two values joined by a derivation.

Relation (3) is what makes the others add up. Reading a note and being an Identity are two facts about two different secrets, and a gate that proved them separately would say nothing about their owner: a thief holding a stolen payload could supply the reading half with the stolen key and the Identity half with its own registered Identity, both true, neither joining them. Stating the tie as a relation over shared private witnesses, rather than inferring it from two proofs that happen to share a public point, is what closes that gap.

Folding pays a second dividend, unbudgeted. The three checks it replaces shared a blinded point P_I , and that point existed only so a sigma and a membership SNARK could agree about a value neither revealed. One proof needs no such point – so there is no P_I , no blind, and no witnessed tie point for a prover to choose freely, which was the first of review finding 5’s three defects. The second and third go the same way: every witnessed limb now enters a gadget that range-checks it, and A1 folds its sums of points into sums of *scalars* before multiplying ($eEnc.C = (m_{rec} + tk)G$, never $M_{rec} + (tk)G$), so it performs no curve addition and cannot be driven into a doubling case. A2 cannot fold its sums that way – its plaintext is an arbitrary point – so it *enforces* the incomplete gadget’s precondition instead, asserting at each addition that the two x-coordinates differ.

No mint-time knowledge helps a non-recipient:

- The issuer knows its own randomness but cannot satisfy (2): that needs a registered account bound to M_{rec} , which only the real recipient has (KYC-gated).
- An attacker who copies ρ cannot spend either; the gate binds to the recipient *Identity* and its registered key, not to ρ .
- A thief holding the whole payload, and so k , still fails (2) and (3).

A1 and A2 differ only in whether M_{issuer} is in the clear (A1) or encrypted for the recipient (A2). Neither affects who can spend.

10.2.1 Identity-Pinning and Recovery

Authority is the *Identity*; the mailbox key is separate. Spending an addressed note requires the receiving secret k that opens the note *and* any registered account bound to M_{rec} . Three consequences, and the first two are why the separation earns its keep:

- **Disclosure is not a key.** A counterparty learns m_{rec} whenever a receipt names the recipient, by design. It buys them nothing further: the Identity scalar decrypts nothing anywhere in the system, so it cannot read the recipient’s notes and cannot test public calldata for them. The cost of naming someone is naming them, and it stops there.
- **Lost account keys are recoverable.** Because *any* Fountain account bound to M_{rec} can redeem, losing one account’s sk does not strand the note: re-register a fresh account on the same Identity (the KYC issuer re-certifies the real person) and deposit. Authority never lived in an account key, and still does not. The reading key k is derived from the wallet’s seed material, so it survives device loss as every wallet secret does, and guardian-wrapped payload copies cover the cases that warrant more. So a note survives key rotation and device loss.
- **No PII-grind shortcut.** Guessing m_{rec} from a person’s details enables nothing. Spending needs the receiving secret, a *registered* account bound to M_{rec} , and a leaf pairing the two, and an attacker cannot make the KYC issuer certify someone else’s Identity. The non-grindable-identities assumption (H a random oracle) and the KYC gate close the shortcut twice over.

So the recovery anchor for *authority* is "prove your KYC Identity again" rather than "preserve one mint-time account key forever", while the anchor for *reading* is the wallet seed, where reading keys belong. A deployment may offer both: an identity-pinned note (recoverable, the default) and an account-pinned opt-in (terminal, for maximal self-custody). At the system level demurrage is the final backstop – value in genuinely-lost notes decays back to the commons rather than being stranded as seigniorage.

Status (on chain). Every gate below is built, set up and verifying a real Groth16 proof on chain, along 32-level membership paths:

Circuit	Constraints	Public inputs	Verifier
deposit_fold_a1.circom	3,309,327	43	DepositFoldA1Verifier
deposit_fold_a2.circom	3,833,335	42	DepositFoldA2Verifier
identity_membership_b1.circom	494,701	9	IdentityMembershipB1Verifier

A2 costs 16% more than A1. Its fifth relation, three curve additions and limb range checks are small; its sixth, the key tie, adds one fixed-base multiplication on H_{PED} beside the seven on G that both carry, and a fixed-base multiplication is what costs. `Notes.spendCoupled{A1,A2}` route through `DepositFoldVerifierAdapter`, which derives EVERY public input on chain and reads the depositor’s registered key and credential from the registry rather than accepting them, so the account relation is necessarily about an account that really is registered. `spendCoupledB1` keeps `verifyDepositorBinding` and composes it with the repaired membership verifier.

The setups are DEV ENTROPY. They are not ceremonies, and nothing built on them should reach a deployment.

10.3 B1 (Bearer): Civil-Trust, Not Cryptographic

The B-spend predicate is a Schnorr PoK of knowledge of ρ plus any valid registered depositor credential. There is no recipient binding: whoever publishes the nullifier first wins the deposit. The issuer, having minted the note, knows ρ ; so does anyone who later observes ρ – including anyone the issuer accidentally leaks it to (database row, printed backup, compromised signing device) and anyone who photographs an unsealed physical cheque.

This is not a cryptographic failure; it is the *point* of a bearer instrument. The B1 flavor replaces cryptographic recipient binding with civil-law recourse:

- The issuer’s Identity M_{issuer} is public on the note. An issuer who prints a cheque and then races to deposit it is visibly on chain as the double-spender. The registered Identity is subpoena-able and the issuer cannot repudiate having printed the note.
- A holder who discovers their B1 has been cashed by someone else has a named counterparty to pursue in civil or criminal court. The chain proves who deposited; the note’s commitment and signature prove who issued it.

The practical rule: **treat B1 like a cashier’s cheque**. Cash within hours to days. Use only issuers whose operational discipline and legal standing you are willing to rely on. Do not use B1 for long-term savings, inheritance, or cold storage; addressed A1 or A2 instruments serve those needs with cryptographic rather than institutional trust.

10.4 Summary: Which Flavors Can Be Saved?

Flavor	Can a non-recipient spend?	Long-term holding safe?	Comparable instrument
A1	No (CP-equality on M)	Yes	Registered bond / named cheque
A2	No (CP-equality on M)	Yes	Named bond, self-wormhole
B1	Yes (anyone who knows ρ)	No	Cashier’s cheque / bearer bond

The consequence: *addressed (A) notes are cryptographically safe to hold; bearer (B1) notes rely on issuer operational discipline and civil recourse*. Placing an A1 or A2 cheque in cold storage for a decade trusts only the cryptography and the recipient’s Identity recovery capability. Placing a B1 in the same vault trusts the issuer not to retain ρ for ten years.

Mapping to use cases:

- **Long-term savings, cold storage, self-issued notes:** A2 – the fully-private self-wormhole between your own encrypted-Identity accounts (the issuer encrypts to your own receiving key, which the wallet seed recovers; the identity scalar does not). A1 if a public issuer Identity is acceptable – for a self-issued note the "public issuer" is simply you.
- **Estate, inheritance, named gifts:** A1 or A2 – A1 when the recipient and heirs want public attribution of the issuer; A2 when discretion is preferred.
- **Payroll, B2C disbursements, audit-visible transfers:** A1, cashed at the recipient's pace.
- **Bearer instruments – cashier's cheques, physical coupons, prize vouchers:** B1, cashed promptly.

11 Physical Security Model

The cryptography only protects what the cryptography touches. The physical paper itself needs a security model. This section sketches one.

11.1 Layered Defense

A printed BUCK Note has three concentric layers of security:

Layer	What it protects	Mechanism
Inner: secret	The cryptographic ability to spend	QR code containing ρ
Middle: physical seal	Bearer (B1) publication-exclusivity	Hologram, scratch-off, laminate sleeve
Outer: identity binding	Addressed (A1/A2) targeting (only named spender)	Encryption under the recipient's receiving key; the folded dep

The *inner* layer is the cryptographic core: as long as ρ (and the flavor-specific material packed into `id-payload`) is unknown, the note cannot be spent. This is unconditional protection against any threat that does not include observing those secrets.

The *middle* layer matters only for bearer B1 notes, where the spend predicate is "any registered holder of ρ ." An unsealed B1 QR photographed by any observer allows that observer to race to spend. Sealing the QR under a tamper-evident covering means the note can be displayed (passed hand-to-hand, stored in a wallet, mailed in an envelope) without exposure. The seal must visibly damage on opening so that recipients can verify they were the first to expose the secret. (This does not rescue B1 from the issuer-race risk – the issuer who printed the note could still have retained ρ before sealing – which is why B1 remains unsafe for long-term holding. The seal addresses post-printing observation, not issuer trust.)

The *outer* layer makes addressed A1/A2 notes safe to *publicly* distribute. An addressed note can be photographed with no security implication: the photographer cannot spend it without also holding a registered credential encrypting the bound `M_rec`. The targeting binding is cryptographic, not physical. For addressed flavors, no physical seal is required.

11.2 Recommended Manufacturing Profile

A practical BUCK Note specification, suitable for industrial-scale printing:

- **Substrate.** Cotton-blend banknote paper or polymer (Bank of Canada-style). Steel or titanium plates for high-denomination "permanent" notes.
- **Printing.** Standard inkjet or laser printing of QR codes is sufficient for cryptographic correctness. For visual anti-confusion (so people can tell a legitimate note from a printout), use UV-reactive ink, microprinting, or watermarks – the same techniques used for traditional banknotes. These do not add cryptographic security; they add *reassurance*.
- **Seal.** Peelable hologram covering the secret QR (bearer B1 only), or scratch-off panel covering a separate code (also B1), or no seal at all (addressed A1/A2, since the cryptography handles exclusivity).
- **Redundancy.** Print the QR twice (front and back) with Reed-Solomon error correction. A note that loses one QR to wear or damage can still be deposited via the second. For higher-value notes, also print the secret as base32 digits in human-readable form.

- **Issuer mark.** A non-cryptographic logo identifying the issuer (the Alberta Buck reserve, a credit union, a commercial issuer). This is for visual recognition only; the cryptographic authenticity check is the on-chain commitment.
- **Denomination indication.** Visible face value for human convenience. The actual value is what the on-chain commitment encodes; the printed face value is metadata. A wallet app that scans an apparent 1,000 BUCK note and reads "actual value: 100 BUCK" from the commitment warns the user before deposit.

12 Use Cases

The same primitive serves a spectrum of use cases that today require entirely separate financial instruments. Each maps to one of the three realisable flavors (A1, A2, B1) according to whether recipient binding and public issuer attribution are required.

12.1 Walking-Around Money

For digital "send at the moment of payment," an *addressed* note targeted to the payee's registered M is ideal: only the holder of that registered Identity can deposit it – *any* account bound to M (the Identity is re-issuable, so long as its scalar $m = H(\text{identity_data})$ is unchanged), with no recipient secret key required beyond being that Identity. In the shipping product this is an **A1** note – the payer's Identity is public and attributable (bound at mint by `verifyIssuerSchnorr`) and exchanged to the recipient through the note payload. The **A2** variant additionally keeps the *payer's* Identity private from third parties (the issuer's Identity is encrypted to the recipient rather than in the clear); it is implemented and exercised end to end, the premium option whose recipient-burden posture is the price of issuer anonymity. For truly paper bearer cash – a note that anyone who holds the paper can spend – a **B1** cheque with hologram seal fills the role, issued by a trusted public party (credit union, commercial issuer) who publicly warrants erasure of ρ after printing.

Most walking-around use cases – paying for a coffee, tipping, splitting a bill with a friend – are served by A1 message notes exchanged at the moment of payment. The paper B1 form is the right choice when one party has no digital wallet at hand or when strict bearer-on-paper semantics are required. Where the payer additionally wants their own Identity hidden from third parties – and is willing to verify the note at delivery under the recipient-burden posture – the premium A2 form applies.

Notes always yield the current pool demurrage rate, so neither encourage nor discourage rapid circulation rather than hoarding. The wallet-app status codes give immediate VALID / DEAD / FAKE feedback on scan. Addressed A1 (and A2) message notes can be redeemed only by the targeted Identity M , so a finder or a copy-retaining issuer cannot spend them. The **B1** paper form is different: it is bearer, so anyone holding ρ can deposit it and the public issuer necessarily knew ρ at mint – B1 notes therefore carry the usual bearer theft exposure and should be deposited promptly.

12.2 Cross-Border Remittances

A paper BUCK Note is just paper. It crosses borders in pockets, envelopes, and parcels at zero marginal cost. An *addressed* note (A1 in the shipping product; the premium private-issuer A2 when the sender's Identity must also stay off-chain) targeted to the recipient's registered M can be redeemed only by that Identity – any account bound to M , no recipient secret key required – so the transfer is useless to anyone else and loss in transit is not a financial risk.

This compares favorably to the existing remittance industry, which extracts roughly 6.5% in fees globally on \$700+ billion in annual transfers.¹³ A paper-note remittance has a fixed cost: print

- mail. The mail might cost \$1; print costs fractions of a cent;

deposit gas costs a few dollars at most. At any reasonable remittance value, the fee compression is enormous.

¹³World Bank. "Remittance Prices Worldwide" (Q4 2024). Average global cost of remitting \$200 was approximately 6.5% in 2024. Total global remittance flows estimated at \$702 billion in 2024.

The recipient does not need a smartphone or internet at the moment of receipt – they need them only at the moment they want to convert the note to on-chain BUCKs or to local fiat. In countries with intermittent connectivity, this is a major usability improvement over digital remittance services.

12.3 Tip Jars and Charitable Donations

A sheet of small-denomination B1 bearer notes (denominated at, say, 2 BUCKs each), perforated for tear-off or printed on small cards, can be generated, printed and kept in their owners' wallets.

Patrons take notes and drop them in the tip jar just like a bill or coin, and the value eventually is deposited and flows to recipient's BUCK account.

This works without any intermediary, payment processor fee, or opening of an account. The patron pays in paper; the recipient receives in on-chain BUCKs. B1 is appropriate here because the turnaround is fast: the tip jar's operator empties the day's BUCK B1 tips to their main account nightly.

12.4 Cold Storage and Disaster Preparedness

A self-issued addressed note (issuer = recipient, both the same registered **M**), stored in a fireproof safe, is a more durable form of cold storage than a 12-word seed phrase. This is an **A1** self-issued note (your own public issuer Identity on the note); the fully-private **A2** self-wormhole, which also hides which encrypted account minted it, is the more expensive and private enhancement.

The note is *self-evidently* an instrument of value – a customs officer, a relative, or your future self immediately recognizes a paper labeled "1,000 BUCK Note" as something worth keeping safe. A seed phrase, by contrast, is opaque to anyone who has not been trained to recognize it.

Furthermore: since the recipient Identity **M** of the Note is cryptographically verified at deposit, there is little risk to printing multiple copies of each A1 or A2 note and storing them in various safe locations – you, and only you as the holder of the registered Identity **M**, can deposit it (any account bound to **M**; no mint-time key to preserve). As soon as you do, the remaining copies are void, and can be discarded.

For disaster preparedness: a small folder of BUCK Notes in a fireproof safe survives infrastructure failures that would render exchange accounts and even hardware wallets unusable. As long as the chain is reachable somewhere in the world, the Notes can be deposited. In a true grid-down emergency, **B1** bearer notes circulate hand-to-hand at face value (addressed A-notes stay redeemable only by their target **M**, so they protect stored value rather than circulate), with the pool-rate demurrage clock running uniformly, so prices remain stable in BUCK terms even during a multi-week outage.

A1 (issuer = recipient) is the natural choice for self-directed cold storage, with A2 the more private self-wormhole; A1 or A2 targeted to a designated heir's **M** is the choice for estate-directed cold storage. The cold-storage bundle need only contain the note opening: an A-note is bound to the recipient *Identity* **M**, not to any mint-time key pair, so it stays spendable under Identity re-issuance – the durable secret is the identity scalar $m_{\text{rec}} = H(\text{identity_data})$, recoverable from the holder's backed-up `identity_data`, and *any* account re-registered on the same **M** can deposit it (see *Long-Term Holding Safety: Identity-Pinning and Recovery*). Practically, the safe-deposit box or heir-accessible vault holds the sealed commitment opening; spend authority travels with the holder's recoverable Identity, with no mint-time private key to lose.

12.5 Estate Planning

A2 notes targeted to your heir's registered M, stored in a safe-deposit box: zero probate process, zero tax-event delay, zero court intervention. The heir scans on your death and deposits with their *own* registered Identity – no key, seed, or secret inherited from you is needed, only the note opening and the heir's own M (any account bound to it, re-issuable) – and the BUCKs are theirs. A1 is the same instrument with the issuer publicly attributable – appropriate where the heir or executors need the origin on-chain for tax or legal reasons.

For larger estates, additional conditional-spend predicates (time locks, death-certificate oracle attestation) can be composed into the commitment's `predicate` field. The SNARK does not care what the predicate is as long as it is well-defined at mint time.

12.6 Gift Cards Without Issuer Risk

A traditional gift card carries the risk that the issuing merchant goes bankrupt between purchase and redemption (cumulative consumer losses to bankrupt gift-card issuers run into the hundreds of millions annually in North America¹⁴). An addressed A1 or A2 BUCK Note carries no issuer risk: the value is on chain in the BUCK reserve, not on the merchant's books.

Specialized merchant gift cards become unnecessary – replaced by BUCK Notes of appropriate denomination, optionally bundled with a paper merchant receipt indicating the intended use.

12.7 Tax-Compliant Compensation

An employer paying a contractor in BUCK Notes faces the same tax-reporting obligations as paying in cash: payment is income and must be reported. The on-chain issuance trail makes this easier to comply with than paper cash, not harder – the employer's mint transaction is an auditable record of the payment, even though the contractor's deposit is private.

For employees: employers can issue A1 notes targeted to each employee's registered Identity, automating payroll while preserving employee privacy from external observers.

12.8 Underbanked and Cash-Economy Inclusion

This is, perhaps, the most important use case.

The Bank of Canada estimates 1-2% of Canadians are unbanked, with rates an order of magnitude higher in rural and Indigenous communities.¹⁵ Globally the World Bank estimates 1.4 billion adults are unbanked.¹⁶

For these populations, paper cash is not a privacy preference; it is the only viable financial instrument. A digital-only currency (CBDC, stablecoin, traditional e-payment) excludes them by definition. BUCK Notes includes them: a person without a bank account, without a smartphone, without literacy in the local language, can hold and transfer BUCK Notes by physical handover, exactly as they hold and transfer paper currency today.

When and if they choose to interact with the digital economy, they deposit a note and the value moves on chain. Until then, they participate in the BUCK economy on exactly the terms they participate in the cash economy. The issuer of the paper – a credit union, a reserve-backed issuer

¹⁴Consumer Federation of America. "Gift Cards: A Costly Convenience" (2023). Estimates of cumulative consumer losses to bankrupt gift-card issuers in the United States range from \$200M-\$1B annually depending on methodology.

¹⁵Bank of Canada. "2022 Methods-of-Payment Survey" (2023). Approximately 1-2% of Canadians lack bank accounts; rates substantially higher in rural and Indigenous communities.

¹⁶World Bank. "The Global Findex Database 2021" (2022). An estimated 1.4 billion adults globally remain unbanked, with disproportionate concentrations in low-income countries, women, rural populations, and the elderly.

like ATB, or a local community organisation – takes on the role that the central bank currently plays for physical fiat, while the recipient retains the anonymity of a cash user.

13 Privacy: Defeating Mass Harvest, Preserving Bilateral Disclosure

BUCK Notes do not make transactions anonymous. They do not hide who paid whom from the parties themselves: the mutual-decryptability invariant means the issuer and the eventual depositor always learn each other's M . They also do not hide a specific transaction from a properly targeted investigation: a warrant served on either named party to a Note compels disclosure of the counterparty's M and the surrounding payments.

What Notes do is *break the on-chain bulk-harvest channel*. Aggregate transfer graphs, KYC-provider subpoenas scoped across thousands of clients, and commercial chain-analytics fingerprinting all operate by correlating on-chain addresses into behavioural profiles without a specific target. Notes replace the on-chain address-to-address edge with a mint/deposit pair separated in time and dissociated by the pool, leaving the bulk observer only the deposits made *directly to or from that observer's own clients*. Everything else becomes a commitment landing in the pool and a nullifier leaving it.

13.1 What the On-Chain Observer Loses

Consider an analytics firm that purchases KYC-linked address data from a large wallet provider. Today, such a firm can reconstruct much of a BUCK user's economic life: regular payroll deposits, recurring vendor relationships, counterparties' other clients, timing patterns. Under Notes, the on-chain trace of a typical legitimate transaction is:

1. Payer's account transfers BUCKs to the Notes contract (mint). The observer sees only that *some amount* entered the pool.
2. Some time later – minutes to months – a recipient deposits a note. The observer sees only that *some amount* left the pool to *some* registered account.

There is no observable edge between payer and recipient. The payer's account activity shows only "sent N BUCK to pool." The recipient's account activity shows only "received M BUCKs from pool." Correlating the two requires either (i) complicity of one of the two parties, or (ii) subpoenaed disclosure by either party.

13.2 What Bilateral Parties Keep

The BUCK Identity invariant – that both endpoints of any value transfer can decrypt each other's M – is preserved. The payer learns the recipient's M when they read the recipient's IdentityRegistry entry to construct E_{note} . The recipient learns the payer's M when they verify the note (from m_{issuer} for A1/B1, from the paired $E_{\text{issuer-for-rec}}$ ciphertext for A2).

Neither party sees the other's wider on-chain activity from the Note alone. The payer sees only the specific M of the recipient, not the recipient's other counterparties or balances. The recipient sees the same narrow slice of the payer's life.

13.3 The Mass-Harvest Threat Model

The threat Notes defeat is *undifferentiated bulk observation*:

- A KYC-provider subpoena scoped to "all transactions of clients X, Y, Z for the last five years" yields only direct pool-to-client and client-to-pool transfers. It cannot reconstruct who X paid, or who paid Y, or whether X and Y ever transacted.
- A chain-analytics firm selling behavioural fingerprints cannot build the edge graph at all; there are no edges to cluster.
- An insurer, employer, or landlord who gains read access to a counterparty's on-chain address learns that address's pool-inflow and pool-outflow amounts, not its counterparties.

The threat Notes *do not* defeat is *targeted investigation with proper authorization*:

- A warrant served on party X to a specific Note compels X to reveal the counterparty's M and the associated context.
- A criminal probe that identifies a "mule" account depositing many Notes can compel that mule to disclose each note's issuer M (the income source) and each outgoing note's recipient M (the onward payment), walking the graph one cooperating party at a time.
- Sanctioned or frozen Identities are blocked at the contract level regardless of Note use.

This is the posture of a bank cheque or personal cheque, not a \$20 bill: the parties to any specific instrument are identified to each other and compellable on individual warrant, but no omnibus order reaches across the system.

13.4 The Regulatory Audit Path

A regulator investigating illicit flows has a clean single-target path:

1. Identify a suspect account (by KYC on a wallet provider, a prior investigation, or a direct complaint).
2. Compel the suspect's wallet, via warrant, to disclose the plaintext M of every counterparty the suspect has transacted with via Notes, together with the note metadata (issuer / recipient, amount, time). This is entirely derivable from material the suspect already possesses: their own registry keys and the payloads they decrypted at scan or mint time.
3. Serve the next warrant on the resulting named counterparty. Repeat.

The audit is depth-first on named parties, not breadth-first across the chain. The *origination* of any specific note remains traceable through the on-chain mint record plus the issuer's own records; a bearer \$20 bill has no analogous trail.

13.5 What Compliance Looks Like

A reasonable regulatory regime around BUCK Notes:

- *Issuers* must be identity-registered and may face reporting obligations on large issuances (analogous to current cash-withdrawal reporting).
- *Depositors* face the reporting obligations that attach to cash deposits at a bank today.

- *Holders* who pass notes hand-to-hand have no reporting obligation, the same way there is no obligation to report carrying \$200 in a wallet.
- *Named parties to a specific note* are compellable to disclose that note’s counterparty and surrounding context on warrant.
- *Pool-rate demurrage* makes long-term cash hoarding economically unviable, bounding the value of Notes-as-store-of-wealth for illicit actors.

The regime is strictly more privacy-preserving than the current surveilled-electronic-transfer regime (which exposes every edge to aggregators and KYC providers) and strictly more compliance-friendly than the current paper-cash regime (which exposes *no* issuance trail at all, regardless of who is investigating).

13.6 The CBDC Comparison

China’s e-CNY, the European Central Bank’s proposed digital euro, the Bank of Canada’s CBDC research – all of these designs include either explicit or implicit surveillance capabilities. The central bank can see, freeze, expire, restrict, geofence, or retroactively examine any holder’s balance.²

This is not an accident of the technology; it is a design goal. The pitch to governments is precisely that CBDCs give them visibility into civilian payment flows that paper cash denies them.

BUCK Notes is the opposite posture. Visibility is at the edges of a single instrument (the two named parties to a specific note), never across the system. The bearer-circulation phase of B1 notes is private in the way cash is private; the addressed A1/A2 phase is private between the two named parties the way a sealed letter is private. A jurisdiction adopting BUCK Notes gets better per-instrument compliance tools than paper cash provides today without enabling the population-scale surveillance a CBDC would.

14 Comparison With Prior Art

This section places BUCK Notes in the lineage of digital cash and privacy-preserving pool designs from 1982 to the present, and identifies what each predecessor lacked.

14.1 Chaum's E-Cash (1982-1998)

David Chaum's blind-signature scheme⁴ introduced the core idea: an issuer (a bank) signs a token blinded by the holder so that the bank cannot link the issuance to the redemption. When the holder spends the token, the merchant verifies the signature without revealing who originally received it.

This is a beautiful design and operationally similar to BUCK Notes in spirit. Its failures were *institutional*, not cryptographic:

- Required a centralized issuer (the bank)
- Required online verification at spend time (no offline transferability)
- Two-sided market never bootstrapped (no merchants accepted, so no consumers held)
- DigiCash (the commercial implementation) went bankrupt in 1998

BUCK Notes has the same privacy properties without the centralized issuer (any identity-registered party can mint notes against their own on-chain BUCK). Spend-time verification is on chain rather than via the issuer (so the issuer cannot selectively block redemptions). The two-sided market problem is mitigated by BUCK's existing on-chain ecosystem – BUCK already exists as a usable currency, so adopting BUCK Notes is a UX upgrade rather than a greenfield bootstrapping.

14.2 Brands' Offline Divisible E-Cash (1993)

Stefan Brands' scheme⁵ addressed the offline-spend problem: a divisible coin can be spent in pieces without contacting the issuer, but *double-spending mathematically reveals the spender's identity*.

This is the "deterrent rather than prevention" model. It works in theory but has two operational issues:

- The deterrent only deters if identity revelation has consequences (it relied on a legal/social enforcement layer external to the cryptography).
- The complexity of divisible-coin management was substantial and never produced a shipped product.

BUCK Notes handles double-spend by *prevention* (global nullifier set), not deterrence. No legal layer is required. Divisibility is achieved by issuing multiple notes at mint time rather than by a single divisible coin.

14.3 Casascius Coins (2011-2013)

Mike Caldwell's Casascius coins³ were physical Bitcoin: a brass or silver coin with a holographic seal covering a private-key QR code. Snap the seal, scan the QR, the coin's Bitcoin balance was yours.

Casascius coins remain the closest existing analogue to paper BUCK Notes. What they lacked:

- Native integration with the underlying token (the Bitcoin protocol does not understand bearer-coin form; the coin is an aftermarket wrapper).
- Targeting (all coins were pure bearer; no way to issue a coin claimable only by a specific recipient).
- Demurrage (a Casascius coin holds its full Bitcoin balance forever, unrelated to any economic mechanism).
- Programmable predicates (only one mode: bearer).
- Issuer was a single individual, who eventually stopped issuing in 2013 due to FinCEN money-transmitter concerns – an issuance bottleneck that BUCK Notes distributes to all identity-registered participants.

BUCK Notes is, in essence, what Casascius would have been if Bitcoin had been designed with bearer-paper as a first-class instrument from the start, and if the underlying token had a demurrage and identity layer.

14.4 BIP-38 Paper Wallets

Encrypted Bitcoin private keys printed on paper, with a passphrase needed to spend. BIP-38 paper wallets are an aftermarket bearer instrument similar to Casascius but without the physical coin form factor.

Same limitations as Casascius (no native integration, no targeting, no demurrage, no predicates), plus the additional UX burden of remembering or transmitting the passphrase.

14.5 Zerocash / Zcash (2014-)

Ben-Sasson, Chiesa, Garman, Green, Miers, Tromer, and Virza’s Zerocash⁷ gave us the first scalable on-chain commitment-and-nullifier UTXO model with full zero-knowledge. Zcash deployed it in 2016.

Zcash provides on-chain shielded transactions but does *not* extend the model to physical bearer instruments, targeted cheques, or identity-linked issuance. All Zcash value lives on chain in shielded notes. No mechanism exists for printing a shielded Zcash note as a paper instrument that someone else could deposit later.

BUCK Notes uses Zerocash’s commitment-and-nullifier primitive for its on-chain machinery and extends it with the Identity-binding layer and the physical-paper surface. The contribution is in the extension, not the primitive.

14.6 Aztec / Zk.Money

Aztec’s predicate-encrypted notes⁸ generalize Zerocash to private smart contract calls – a note can encode arbitrary state operated on by arbitrary predicates. This is the cryptographic ancestor of the **predicate** field in the BUCK Note commitment.

Aztec does not have physical bearer instruments either. Its predicates are designed for on-chain DeFi applications (private DEX trades, private lending), not for paper notes or identity-bound cheques. BUCK Notes applies the same predicate machinery to a different problem: specifying spend conditions for a private bearer instrument that is issued against an identity-registered reserve.

14.7 Tornado Cash and Privacy Pools

Tornado Cash⁹ was the simplest possible commitment-and-nullifier mixer: fixed-denomination deposits in, anonymous withdrawals out. Sanctioned by OFAC in 2022 for money-laundering use.

Privacy Pools (Buterin, Schaefer, Tromer, et al., 2023)¹⁰ extends Tornado with per-spend "association sets" – a depositor can prove their deposit is in a non-incriminating subset, allowing compliance without identification.

Both are mixer designs, not cash-instrument designs. Their relevance to BUCK Notes is the lineage: the Note Pool is a Tornado-style global anonymity set, but with predicates and Identity bound in. The selective-disclosure compliance story borrows from Privacy Pools' association-set ideas, but with issuer identity substituting for the depositor-declared non-incrimination proof.

14.8 Comparison Summary

System	Bearer paper	Targeting	Demurrage	Predicates	Native	Decentralized	Year
Chaum E-Cash	No	No	No	No	-	No	1982
Brands Offline	No	No	No	No	-	No	1993
BIP-38 Paper Wallet	Yes	No	No	No	No	Partial	2010
Casascius Coin	Yes	No	No	No	No	No	2011
Zerocash / Zcash	No	No	No	No	-	Yes	2014
Tornado Cash	No	No	No	No	-	Yes	2019
Aztec	No	No	No	Yes	-	Yes	2020
Privacy Pools	No	No	No	Limited	-	Yes	2023
CBDCs (e-CNY, etc.)	No	Yes	Sometimes	No	Yes	No	2020+
BUCK Notes	Yes	Yes	Yes	Yes	Yes	Yes	2026?

The "Native" column captures whether the bearer-paper instrument is a first-class designed primitive of the system (as opposed to an aftermarket wrapper or absent entirely). BUCK Notes is the first design that fills every column. This is not because the prior art was incompetent; it is because the four required pieces (commitment-pool primitive, identity layer, demurrage, jurisdictional reserve) finally converged in one system.

14.9 Comparison: Cash, CBDC, BUCK Notes

A more direct comparison aimed at a non-cryptographer reader: how does BUCK Notes stack up against physical cash and central bank digital currencies?

Property	Physical Cash	CBDC (typical)	BUCK Notes
PRIVACY			
Holder's identity hidden	Yes	No (issuer sees all)	Yes (during circulation)
Transaction graph hidden	Yes	No	Yes
Issuer's identity hidden	Yes (notes are fungible)	No	Only to bulk observers; visible bilaterally
FINALITY			
Settlement is instant	Yes	Yes	Yes (chain finality)
Reversible by issuer	No	Yes (CBDC freeze)	No
Reversible by holder	No	Yes (chargeback)	No
INFRASTRUCTURE			
Works without internet	Yes	No	Yes (transfer); needs net for deposit
Works without battery	Yes	No	Yes (paper); needs power for deposit
Works without bank account	Yes	No	Yes
Cross-border in your pocket	Yes (with declaration)	No (geofenced)	Yes
DURABILITY			
Survives infrastructure failure	Yes	No	Yes (paper); chain on recovery
Counterfeit-resistant	Partial (printed security)	Yes	Yes (cryptographic)
Recoverable if lost	No	Yes (issuer can re-issue)	A-notes yes (re-register on same Identity); B1 no
ECONOMIC BEHAVIOR			
Lost notes extracted forever	Yes	No	No (returned via demurrage)
Inflation tracks issuer policy	Yes	Yes	Yes (tracks BuckK)
Holds value over decades	No (inflation)	No (inflation)	No (demurrage)
Holds value over years	Approximately	Approximately	Yes (with demurrage discount)
PROGRAMMABILITY			
Time-locked spends	No	Possible	Yes
Conditional payouts	No	Possible	Yes
Multi-signature spending	No	Possible	Yes
Issuer can restrict spending	No	Yes	No (issuer relinquishes control at mint)
COMPLIANCE			
Issuance is auditable	No	Yes	Yes
Circulation is auditable	No	Yes	No (private)
Deposit is auditable	Partial (CTR rules)	Yes	Yes
Selective disclosure available	No (no records)	No (full disclosure)	Yes

The dominant insight from this table: *BUCK Notes combines the privacy properties of physical cash with the compliance and durability properties of CBDC, while preserving infrastructure independence that CBDC abandons.*

The single property where physical cash is genuinely irreplaceable – "works without internet for deposits" – is moderated for BUCK Notes but not eliminated. Holders can transfer BUCK Notes hand-to-hand offline, indefinitely; they need internet only when they want to convert the bearer instrument to an on-chain balance. In a disaster scenario, BUCK Notes circulate as cash for the duration of the outage, then deposit normally when the chain becomes reachable again.

15 Required Identity Cryptography

BUCK Notes re-use the existing Identity machinery (Identity) without introducing new hardness assumptions. The whole construction is built from primitives already proved sound in the Proofs document. The only new moving parts are two standard SNARK-side primitives (Poseidon hash, Poseidon-based nullifier PRF) whose role is internal to the circuit.

15.1 Components Already Present

The following are unchanged from the identity layer and are used directly:

1. **Pointcheval-Sanders issuer signature** σ on the Identity scalar m . Re-randomization continues to serve unlinkable credential generation (Identity Fountain). No change.
2. **ElGamal registered credential** $E = (R, C) = (rG, M + r \cdot pk)$ where $M = m \cdot G$. Continues to be the on-chain identity witness. At mint the issuer *addresses the recipient Identity* M_{rec} and keys the payload to that Identity’s registered receiving key – encrypting its own M_{iss} under pk_{recv} (A2), or naming it in the clear (A1) – rather than re-randomising the recipient’s account credential.
3. **Chaum-Pedersen equality of discrete logs (Theorem 6)**. Used verbatim by the A-spend predicate to prove that the depositor’s *current* registered credential decrypts to the same M that E_{note} encrypts. This is the same proof the **approve** flow produces outside a SNARK; in Notes it is embedded in the spend circuit.
4. **Schnorr signature on cm** . Used in the mint circuit to bind A1 and B1 notes to their public issuer. Standard construction.
5. **Schnorr PoK of sk for registered pk** . Used by the B-spend predicate to bind the depositing account to a registered identity. Single-statement case of CP.
6. **IdentityRegistry** mapping between on-chain addresses and registered credentials (PS signatures, ElGamal ciphertexts, isPublicIdentity flags set by **bindContract**).

These six components carry two distinct kinds of authority, and the note layer adds a third. The identity scalar $m = H(\text{id})$ (from which $M = mG$ and the ElGamal credential are derived) is what a receipt discloses: it names a counterparty. It is handed over by design – Alice learns Bob’s m when a receipt names him. It does not decrypt notes. The receiving secret k does, and it is derived from the wallet seed, never from m . The PS signature σ grants *write* capability: register accounts. It is held only by its owner. A counterparty who knows m can name the Identity, cannot read its notes, and cannot impersonate it.

Notably absent: a stealth-address construction is *not* used by Notes. Recipient binding is the receiving key, exchanged on the same out-of-band channel as M_{rec} , and proved at spend by the folded gate rather than by a fresh key per note. Whether one mailbox per recipient is enough, or one per payer, is open: a single key does not link notes under DDH, and a leaked k reads every note still addressed to it.

15.2 New Primitive: The Note Commitment Hash

A SNARK-friendly commitment hash H_{cm} (Poseidon) is needed to compute the commitment inside the spend circuit. The hash is deterministic, collision-resistant over the note’s field, and efficient in

R1CS; Poseidon over the BN254 scalar field is the natural choice and is already used by the Aztec and Zcash Sapling circuits.

No novel cryptographic construction; only a circuit-level primitive that is standard in modern SNARK systems.

15.3 New Primitive: The Nullifier PRF

The nullifier PRF is Poseidon-based and is the *same* derivation for every spend shape (the shipped `spend.circum`): $nf = \text{Poseidon}(\rho, idHash, 4242)$, the constant tag separating nullifier preimages from commitment preimages. (A second tag, 4243, is reserved in the wallet helpers for a future flavor-split derivation; the shipped circuit does not use it.)

What differs per spend shape is the *authorization* around the nullifier, not its derivation:

- **A-spend:** the folded deposit gate requires the depositing account to be bound to the recipient identity m_{rec} , and re-derives nf in-circuit from $(\rho, idHash)$ in the same proof, tying the gate to the specific spent note. One statement does both, which is the point: the note tie and the account relation share the witness they are about instead of meeting at a public point. Authorization keys on the *identity* M_{rec} , not on any particular account key-pair: the nullifier is independent of which bound account performs the deposit (multiple credentials for one M cannot yield distinct nullifiers and double-deposit), *and* key loss is recoverable – a recipient who loses an account key binds a new account to the same m_{rec} and spends. Reading is the one thing an account key never carried: that is the receiving secret, held in the wallet seed, where reading keys belong.
- **B-spend:** knowledge of the opening alone authorizes; the nullifier is publicly computable by anyone who knows $(\rho, idHash)$. This is the first-come-first-served property of bearer B1 notes.

Because the issuer chooses ρ and computes $idHash$ at mint, she can pre-compute nf and watch for the note’s redemption – the non-deniable-receipt / anonymity-cost posture documented under *Use Cases*. Watching is not spending: passing the A-spend deposit gate still demands m_{rec} .

15.4 Encryption at Mint – Under the Recipient’s Receiving Key (A2)

For A2 the issuer encrypts its *own* registered Identity under the recipient’s receiving key:

$$eIss = (R_e, C_e) = (r'G, M_{\text{issuer}} + r'pk_{\text{recv}}),$$

decryptable by k , not by m_{rec} . The value ciphertext is keyed the same way. A blinded re-encryption proof (`verifyIssuerReenc`) binds $eIss$ to the issuer’s *registered* Identity without revealing it (anti-framing). For A1 the issuer publishes M_{issuer} in the clear with a Schnorr signature, and encrypts the plaintext M_{rec} under the same pk_{recv} . For B1 there is no mint-time recipient ciphertext.

15.5 Deposit Gate (A1/A2) – One Folded Proof

The depositor proves `deposit_fold_a1` or `deposit_fold_a2`: one witness, the relations in *A-Flavors (Addressed)* above. The circuit hashes $\text{Poseidon}(\tau_R, m_{\text{rec}}, k, salt)$. It does not publish a blinded point P_I , and A1 performs no curve addition. A2 does, and it asserts the incomplete adder’s precondition at each addition. The on-chain adapter derives the public inputs and reads the depositor’s registered account from the registry.

15.6 Deposit Gate (B1) – Depositor Binding (the A2 dual)

The depositor encrypts its *own* Identity under the public issuer key, $E_{\text{dep} \rightarrow \text{iss}} = (rG, M_{\text{depositor}} + rpk_{\text{iss}})$, and proves (**verifyDepositorBinding**, EIP-196) that it encrypts a registered Identity bound to its payout account. The issuer decrypts the deposit event and names the depositor. Collusion-resistant by the shared witness coupling "account bound to M " to "ciphertext encrypts M ".

15.7 Summary: Cryptographic Components

Component	Role in Notes (Identity-M)	Assumption
Poseidon commitment / nullifier	note opening + double-spend PRF	ROM-like
Poseidon-Merkle identity accumulator	" M is a registered Identity" membership	CR (Poseidon)
ElGamal on G_1 (under a <i>receiving</i> key)	encrypt the counterparty Identity	DLog / DDH
Folded deposit gate (Groth16, A1/A2)	hidden-Identity spend, one witness	system-dep
Okamoto sigma (B1 depositor binding)	hidden-Identity deposit gate, one secret	DLog + ROM
Hash-to-curve Pedersen generator	B1's P_{dep} , opened by two proofs	unknown DLog
Blinded re-encryption (verifyIssuerReenc)	A2 anti-framing at mint	DLog + ROM
Schnorr signature (public issuer A1/B1)	name the public issuer at mint	DLog + ROM
Verifiable decryption (Chaum-Pedersen DLEQ)	the unilateral receipt	DLog + ROM
Groth16 (membership) / batch-mint SNARK	membership + the rolling-tree mint	system-dep

No novel hardness assumption is introduced, but one is now *named* where it was silently assumed. B1 publishes P_{dep} and opens it with two separate proofs that must agree; that only holds if nobody knows $\log_G H$. A generator built as $\text{keccak}(\cdot)G$ has a public one, and a depositor holding any registered identity scalar m' – which counterparties hold by design – could shift the blind by $(m_{\text{dep}} - m')/h$ and spend while unregistered. B1's generator is therefore hashed to the curve, under **AlbertaBuck/Pedersen/H/v2**.

A2 rests on the same assumption for a different reason: the fold opens the mint binding's T again, so its blind must be on a generator with no known logarithm, or the blind absorbs any difference of Identities and the key tie binds nothing. A1 has no point opened twice and needs neither. The receipts compose a blinded re-encryption (or the public issuer's signature) with a verifiable decryption. Full proofs in alberta-buck-proofs.org; the identity-axis unification in the "one gadget" and Mutual Decryptability sections here and the companion *Proofs*.

16 The SNARK Circuits

The Notes implementation uses three BN254 Groth16 circuit families:

- `mint_batch.circom` – the rolling-tree batch mint, parameterised by pinned `N`.
- `spend.circom` – the flavour-bound note proof (the rolling-tree opening), shared by all three coupled spends.
- the **deposit gate**, whose shape differs by flavour because the number of secrets its claims rest on differs. `deposit_fold_a1.circom` and `deposit_fold_a2.circom` prove the addressed gate in ONE statement over one witness; `identity_membership_b1.circom` proves B1’s membership beside the cheap EIP-196 Okamoto sigma `verifyDepositorBinding`, which B1 keeps because its two claims share a secret and its nonce therefore ties them.

That split is the architecture, not an implementation accident. A sigma can tie two claims only when they are about the same witness; where they are about two – an Identity and a receiving key – the tie has to become a relation, and a relation needs a circuit.

16.1 Mint Circuit (`circuits/mint_batch.circom`)

A per-leaf on-chain Poseidon insertion would be gas-cost-fatal at realistic batch sizes ($\sim 800K$ gas/leaf; $N=16$ alone exceeds the block limit), so `mint_batch.circom` folds the Merkle-tree update *inside the SNARK*: the circuit consumes `oldRoot` as a public input, folds N commitments onto it using witness sibling paths, and exposes the resulting `newRoot` as a public output. On-chain mint cost becomes essentially constant in $N - \sim 380K + \sim 500$ gas/leaf of calldata.

Public inputs (5 entries, N -independent).

- `oldRoot` – the live note root the prover read from chain
- `newRoot` – the root after folding `cm_1..cm_N`
- `nextLeafIndex` – the live tree size the prover read from chain
- `totalFace` – $\sum v_i$, range-bounded to 128 bits
- `cmBatchHash` – $H(cm_1, \dots, cm_N)$, letting the contract bind calldata to the SNARK without making every `cm_i` its own public input (keccak on chain; Poseidon reserved for a future in-field variant)

Witnesses. For each leaf i : (`flavori`, v_i , ρ_i , `idHashi`, `predHashi`) plus the `TREE_DEPTH` sibling hashes `pathSiblings_i[0..19]` needed to fold `cmi` into the rolling tree.

Constraints.

1. `totalFace` = $\sum v_i$ (amount conservation).
2. Each `cmi` = Poseidon5(`flavori`, v_i , ρ_i , `idHashi`, `predHashi`) (opening binding).
3. Each $v_i \in [0, 2^{128})$ and `totalFace` $\in [0, 2^{128})$ via `Num2Bits(128)`.
4. Rolling Merkle insertion: threading `oldRoot` and `nextLeafIndex` through N successive single-leaf insertions (Switcher + Poseidon-T3 per level, per leaf), the result equals `newRoot`. About 4.3K constraints per leaf; total $\sim 6K$ R1CS per leaf including the opening.

5. Calldata binding: $\text{cmBatchHash} = H(\text{cm}_1, \dots, \text{cm}_N)$.
6. Belt-and-suspenders: $\text{cm}_i \neq 0$ and $\text{cm}_i \neq \text{ZERO_VALUE}$ per leaf.

Pinned N set. Each pinned N has its own compiled circuit, trusted setup, and deployed verifier. Recommended initial set: $\backslash 16, 128, 1024 \backslash$. The wallet rounds any requested split up to the smallest pin $N^* \geq N_{\text{requested}}$ and pads with $v = 0$ dummy openings; dummies are valid commitments in the tree but spendable for zero BUCKs.

Contract side. The Notes contract checks `oldRoot` and `nextLeafIndex` against its live state, recomputes `cmBatchHash` from the calldata `cms[]`, dispatches to `mintVerifiers[cms.length]` to verify the proof, pulls `totalFace` BUCKs from the caller via the standard ERC-20 path, writes `newRoot` into the ring buffer, advances `nextLeafIndex` by `cms.length`, and bumps `noteFaceSum` by `totalFace`. A single `Minted` event covers the whole batch.

16.2 Spend: Note-Opening + the Identity-M Deposit Gate

The note-opening proof below is shared by all three flavors. Its public `flavor` is constrained to the committed opening and supplied by the selected contract entrypoint; the Identity-M deposit gate that follows establishes *who* is spending.

Public inputs. `noteRoot`, `nullifier`, `face`, `recipient`, `chainId`, `flavor`, and `issuanceCommitment`. For B1 the last input is the exact opened note commitment; for A1/A2 it is zero.

Witnesses. $(v, \rho, \text{idHash}, \text{predHash}, \text{leafIndex}, \text{path}[20])$.

Constraints. Eight groups, mirroring the circuit’s section labels:

- **(R)** Range bounds: $v \in [0, 2^{128})$ and $\text{face} \in [0, 2^{128})$ via `Num2Bits(128)` on each, and $\text{face} \equiv v$ (face exposed publicly equals the witness face).
- **(F)** Flavor policy: `flavor` is exactly A1, A2, or B1. Because the contract supplies the entrypoint’s constant, an addressed opening cannot be replayed through the bearer path.
- **(P)** Predicate policy: `predHash` is zero; no nonzero spend predicate is implemented by this circuit version.
- **(C)** Commitment opening: $\text{cm} = \text{Poseidon5}(\text{flavor}, v, \rho, \text{idHash}, \text{predHash})$.
- **(I)** Issuance attribution: `issuanceCommitment` equals (cm) for B1 and zero for A1/A2. This gives only the public bearer flavor an exact mint-to-spend handle.
- **(M)** Merkle membership: walk `path[20]` with `Switcher` selectors and Poseidon-T3 at each level, asserting the result equals `noteRoot`.
- **(N)** Nullifier: $\backslash(\text{nullifier} = \text{Poseidon3}(\rho, \text{idHash},$
 1. $\backslash)$. The constant tag 4242 is the shared spend-nullifier domain.

Reusing the same (ρ, idHash) across flavors therefore deliberately collides; wallet generation uses a fresh ρ per note.

- **(B)** Ghost binding: `recipient` and `chainId` are constrained into the witness via $x \cdot x$ so the proof cannot be replayed against a different recipient or chain (replay protection by Fiat-Shamir on the public inputs).

Contract side. Each flavour’s entrypoint – `spendCoupledA1` / `spendCoupledA2` / `spendCoupledB1` – requires `root` in the recent `roots` ring and `nullifier` unused, verifies this note proof through `SpendVerifierAdapter` (`spendVerifier.verifySpend`, shared by all three), and for B1 requires the revealed commitment’s mint-authenticated issuer to equal the supplied issuer. It then runs the Identity gate below, sets `nullifiers[nullifier] = true`, decrements `noteFaceSum`, and calls `buck.transfer(recipient, face)` – routed through the Carrying path because the pool is `isCarrying`.

The identity gate, by flavour. The opening proof above establishes *that* a valid note is spent. Who may spend it is a second proof, and the three flavours do not share its shape.

- A1 and A2: `deposit_fold_a1.circom` / `deposit_fold_a2.circom`, one witness, as in *A-Flavors (Addressed)*. The note tie (`nullifier`, `idHash`, the ciphertext) is inside that same proof.
- B1: `verifyDepositorBinding` plus `identity_membership_b1.circom`. The sigma’s shared nonce ties the account to the ciphertext because both use m_{dep} . The membership proof opens $P_{\text{dep}} = M + bH_{\text{PEDERSEN}}$ and a salted leaf.

All three share one `spend.circom` note proof. Its public flavor is constrained to the committed opening; for B1 it also exposes the exact mint commitment so `Notes` can authenticate the supplied issuer.

16.3 Circuit Engineering Budget

Constraint counts for the Notes circuits (BN254 Groth16):

Circuit	R1CS (total)	PTAU pow
<code>deposit_fold_a1.circom</code> (the folded A1 gate, 32-level path)	3,309,327	$2^{\{22\}}$
<code>deposit_fold_a2.circom</code> (as A1, + issuer membership and key tie)	3,833,335	$2^{\{22\}}$
<code>identity_membership_b1.circom</code> (B1’s membership, 32-level path)	494,701	$2^{\{20\}}$
<code>mint_batch.circom</code> (N=1) M	21,914	$2^{\{15\}}$
<code>mint_batch.circom</code> (batch; N=2) M	43,701	$2^{\{16\}}$
<code>mint_batch.circom</code> (batch; N=4) M	87,273	$2^{\{17\}}$
<code>mint_batch.circom</code> (batch; N=8) M	174,417	$2^{\{18\}}$
<code>mint_batch.circom</code> (batch; N=16) M	348,705	$2^{\{19\}}$
<code>mint_batch.circom</code> (batch; N=32) M	697,281	$2^{\{20\}}$
<code>mint_batch.circom</code> (batch; N=128) P	$\sim 2,800,000$	$2^{\{22\}}$
<code>mint_batch.circom</code> (batch; N=1024) P	$\sim 22,000,000$	$2^{\{25\}}$
<code>spend.circom</code> (shared flavor/issuer-bound proof; depth-20 Merkle)	$\sim 12,000$	$2^{\{15\}}$

Measured (M) via `snarkjs r1cs.info build/snark/mint_batch_n${N}.r1cs` on the shipped pinned-N set {1, 2, 4, 8, 16, 32}; projected (P) linearly from the N=16 / N=32 baseline ($\sim 21,800$ R1CS per leaf, $\sim$ half of which is the dual Merkle walk that pins `oldRoot` and advances `newRoot` against the same witness siblings). The PTAU power is the smallest 2^k ceremony whose FFT domain accommodates the circuit plus snarkjs setup overhead.

The spend circuit is comparable to Zcash Sapling’s Spend circuit ($\sim 25,000$) and Aztec’s basic note-spend circuit; on-chip proving on a modern smartphone: a few seconds. `mint_batch` at N=16 is a ~ 6 -second proof on a dev laptop (snarkjs); N=32 is ~ 12 s; N=128 is minutes (rapidsnark territory); N=1024 requires a beefy machine and roughly an hour, with the on-chain verification cost still bounded by a single Groth16 verify regardless of N. The $2^{\{25\}}$ ceremony is the ceiling that fixes the largest pinned N – raising it would require a fresh trusted setup.

16.4 On-Chain Gas (measured end-to-end)

The full Note lifecycle is exercised by `test/NotesE2E.t.sol`, which drives one mutually-consistent fixture per flavour (`alberta_buck/test/vectors/e2e/{a1,a2,b1}.json`, built by `scripts/snark/gen_e2e_fixtu` and shipped as Python package data) through *every gate as a real proof*: the batch-mint Groth16, the shared flavor-bound spend Groth16 against the replayed note tree, B1’s EIP-196 depositor-binding sigma pinned to (depositor, chainid=1), and the deposit gate against an `identityRoot` posted into the registry’s ring – one folded proof for the addressed flavours, a sigma plus a membership proof for the bearer. Gas below is the measured cost under solc 0.8.28, optimizer 200 runs (`make nix-match-NotesE2E`).

Per-flavour lifecycle (one mint + one coupled spend, full call gas):

Flavour	mint	spendCoupled*
A1 (addressed, public)	513,223	966,181
A2 (addressed, private)	629,061	981,834
B1 (bearer, public)	513,223	910,260

A1 and B1 share the public-issuer mint path (the batch Groth16 plus the issuer Schnorr), so their mint cost is identical; A2’s mint carries the extra issuer re-encryption binding and its T outputs ($\sim 116K$). The spend spread is set by the per-flavour verifier mix below.

Per-phase verifier cost (isolated external call gas):

Verification phase	A1	A2	B1
note-opening Groth16 (<code>spend.circom</code> , shared)	241,262	241,279	335,084
folded deposit gate (<code>deposit_fold_a1/a2.circom</code>)	514,845	506,837	–
Identity membership (<code>identity_membership_b1.circom</code>)	–	–	253,120
depositor binding sigma (EIP-196)	–	–	131,046

The addressed flavours now show ONE gate line where they showed three. That is the architecture arriving in the gas table: the coupling sigma, the membership proof and the note-binding tie were three checks meeting at a shared public point, and a shared point cannot tie two different secrets – so they became one proof over one witness. It is also cheaper. A1 paid $253,120 + 89,866 + 377,257 = 720,243$ for the three; it now pays 514,845, a saving of about 205K, and A2 saves about the same. Paying less for a stronger statement is unusual enough to be worth saying plainly: the saving is one Groth16 verification and one sigma that no longer happen.

A2’s gate costs slightly LESS on chain than A1’s despite carrying two more relations, because it publishes no face – 42 public inputs against 43, and on-chain cost tracks public inputs rather than constraints. Its extra 16% of constraints is a proving cost, not a verification one.

B1 keeps two lines because its two claims share one secret, so its sigma is a genuine tie and there is nothing to fold; its membership proof is the repaired circuit. B1 pays more in the shared spend verifier because its issuance-commitment public input is nonzero, while A1/A2 pass zero. A2’s remaining premium over A1 sits in the *mint* (the issuer re-encryption binding, $\sim 116K$ – the price of issuer anonymity, see *Why A2 Costs More*).

A1’s binding needs no second ciphertext. An A1 `idHash` commits ($eNote, m_{\text{issuer}}, \sigma$) rather than the A2 layout’s second ciphertext, so the fold pins the note through its VALUE ciphertext instead: `deposit_fold_a1` checks $eNote = (r_n G, (v + r_n k)G)$ with the spend’s public **face** as v , which makes the addressed Identity unique. That is why A1 ships `eNote`’s randomness in the payload and A2 ships the issuer’s r' , naming salt and γ .

16.5 Common Circuit Gadgets

Shared library used by both circuits:

- Poseidon hash (T3 for Merkle pairs; T6/Poseidon5 for commitment opening; T4/Poseidon3 for nullifier)
- Tornado-style Merkle inclusion (Switcher + Poseidon at each level)
- Num2Bits range proofs on field elements

B1's `identity_membership_b1` gate adds one emulated BN254 $\mathbb{G}_1$ point addition (the $P_{\text{dep}} = M + T$ tie, with its precondition enforced and $T = bH_{\text{PEDERSEN}}$ PROVEN rather than witnessed) on top of the native Poseidon-Merkle walk, keeping the heavy *identity* arithmetic out of the EVM: the deposit sigma verifies in $\sim 90\text{K}$ gas (measured full external call; the raw EIP-196 precompile arithmetic is $\sim 36\text{K}$) and the membership is a single constant-gas Groth16 verify ($\sim 253\text{K}$, *On-Chain Gas* above), versus the $\sim +250\text{K}$ R1CS that native $\mathbb{G}_1$ equality inside the spend SNARK would have cost.

17 What This Architecture Rules Out: Pure-Bearer Cryptographic Notes

The design intentionally does not provide a pure-bearer cryptographic note – one whose spend predicate is "anyone who knows the opening can deposit, with no Identity hook at all." This is the closest analogue to a \$20 bill (possession is the entirety of ownership, no registration involved), and it is attractive for the same reasons physical cash is attractive. Three properties make it a poor fit for the cryptographic surface:

1. *Issuer-race is unbounded.* A pure-bearer predicate $\text{know}\rho$ has no cryptographic mechanism preventing a malicious issuer from retaining ρ and racing the holder to deposit. The physical seal on paper makes the *observation* of ρ detectable to the holder, but does not constrain the *issuer* at mint time.
2. *Deposit has no Identity binding.* BUCK's mutual-decryptability invariant and the regulatory audit path both rely on the depositor being a registered Identity. A pure-bearer deposit with no Identity hook breaks this property and forces the system into one of two uncomfortable corners: accept unregistered deposits (losing all compliance), or reintroduce Identity at deposit time (at which point it is just a B1 note).
3. *Three alternative routes all collapse into existing flavors.* *Blind minting* (the issuer does not know ρ at mint time) requires the holder to participate in minting with a blinded commitment – incompatible with "print a stack of notes and hand them out." *Trusted printer* (HSM-attested erasure of ρ) works in principle but reintroduces a single-point-of-trust. *Short expiry* limits the issuer-race window – equivalent to B1 with an additional expiry predicate.

Rather than contort the cryptographic layer to approximate pure bearer, BUCK Notes keeps the physical-world primitive where it belongs: the *tamper-evident seal over a B1 note* is a physical-exclusivity mechanism, and that is an inherently physical property. Cryptography makes the *opening* hard to forge; a hologram makes the *observation* detectable. The two surfaces complement without pretending to be the same primitive.

For use cases that truly require "no identity link at all":

- *Short-lived hand-to-hand circulation* is served by B1 paper notes with tamper-evident seals, printed by a trusted issuer who publicly warrants that they erased ρ after printing. This warranty is an off-chain social/legal claim identical in kind to the trust extended to the Bank of Canada when it issues physical polymer banknotes. A credit union or a commercial issuer fills the role of the central bank for this purpose.
- *Long-term holding* is served by addressed flavors (A1, A2), where CP-equality binds the note to the recipient's M and no hologram seal is needed.

The "walking-around money" digital flow is handled today by an *addressed* note: Alice targets the payee's registered M at the moment of payment, hands over the opening via message or QR, and the recipient deposits into their own account. In the shipping product this is an **A1** note (public, attributable issuer – bound at mint by `verifyIssuerSchnorr`); the **A2** variant (also shipped) additionally hides the *payer's* Identity from third parties, under the recipient-burden posture. Either way the UX changes relative to paper cash – payment now involves a brief digital handshake – but gains a property physical cash lacks: a dropped or intercepted addressed note is useless to anyone but the intended recipient (the addressed binding, not the issuer-privacy, is what provides this).

18 Security Assumptions and Theorems Used

The unified Notes design rests on the same cryptographic foundations as the existing Identity system, plus one already-proved additional theorem:

Primitive / Theorem	Usage in Notes	Assumption
PS signature (issuer credential)	Mint-circuit issuer-binding (all)	q-SDH
ElGamal on G_1	Recipient payload (A1, A2)	DLog on G_1
Theorem 6 (CP re-encryption)	A-spend predicate	DLog + ROM
Schnorr signature verify	Public-issuer binding (A1, B1)	DLog + ROM
Schnorr PoK (sigma protocols)	B-spend depositor binding	DLog + ROM
Poseidon hash/PRF (SNARK-side)	Commitments, nullifiers	ROM-like (circuit)
SNARK proof system (e.g., PLONK)	Wrapping all of the above	system-specific
Merkle tree (noteTree)	Commitment anchor	collision-resistance

No *novel* cryptographic assumption is introduced. The only new primitive not already in BUCK is the Poseidon hash family, which is standard in SNARK deployments (Aztec, Zcash Sapling/Orchard, Starknet, Mina) and well-studied.

18.1 Demurrage Conservation as a Contract Invariant

Because the pool is a regular BUCK account and the Carrying transfer moves only nominal balance (no fee burn, no `totalSupply` change), no Notes-specific demurrage invariant is needed. The system-wide BUCK invariant

$$\sum_{\text{all accounts}} B_X = \text{totalSupply (incl. Jubilee)}$$

holds trivially through every spend: the Carrying transfer moves exactly `face` BUCK from pool to recipient, leaving the sum unchanged. The recipient’s `buckSeconds` apportionment is pure bookkeeping; no balance sum is touched.

The Notes contract only needs to maintain two auxiliary local invariants:

1. `noteFaceSum` ≥ 0 (no negative note face sum).
2. Every `face` appearing in a spend equals the v that appeared in the corresponding mint (enforced by the SNARK via the Merkle inclusion of `cm` binding v , and the circuit’s `face` $\equiv v$ constraint).

Both are trivial. The only BUCK mechanism Notes leans on is the Carrying-transfer path (`Buck._carryingTransfer`, auto-dispatched by the pool’s `isCarrying` flag), itself a simple state-machine operation apportioning the pool’s `buckSeconds` onto the recipient.

19 Open Questions

The architecture above leaves a handful of economic and cryptographic questions worth sharpening before commitment:

19.1 Donation Incidence

Donations to the Notes contract credit it fresh, zero-age BUCK, diluting the pool’s per-BUCK age and therefore reducing the demurrage share every subsequent redeemer inherits. The benefit spreads across all future redemptions rather than accruing to a single lucky first-redeemer. A donation immediately before a large redemption benefits that redemption more (the dilution is weighted by the donation’s size against the pool’s existing balance), but every subsequent redemption benefits in proportion to the lowered age. No special smoothing logic is needed.

Residual donation principal (after all notes drain) is not lost: it sits in `balanceOf(notes)` carrying whatever buck-seconds it has accrued since its last touch, available to subsidise the next generation of mints, or recoverable by governance sweep per Notes-contract policy.

19.2 Stranded Value on Lost Notes

A lost note contributes to `noteFaceSum` indefinitely. The BUCK backing that note sits in the pool, where its share of accumulated demurrage stays in the pool’s `buckSeconds`. Each spend of a live note inherits a slice of those accumulated buck-seconds on its delivered face – so over time the lost note’s implicit demurrage is paid instalment-style by subsequent live noteholders on their next outgoing transfers. At 1%/yr loss and 2%/yr demurrage, the long-term tendency is that the pool absorbs the lost-note demurrage as a small pool-wide overhead.

19.3 Integer-Arithmetic Rounding

The Carrying transfer’s buck-seconds apportionment involves one integer division $\text{carried} = \text{liveBs} \cdot v / \text{poolRaw}$ for the recipient’s inherited share. Truncation rounds the recipient’s inherited fee *down* by at most one buck-second per spend – trivially small (sub-wei demurrage on any realistic balance) and *not* drift-accumulating: the lost fraction never exceeds the pool’s per-spend integral slice. Floor rounding is fine; no banker’s logic required.

19.4 Predicate Programmability vs. Circuit Simplicity

Keeping three fixed flavors simplifies the circuit count. Adding programmable predicates (time locks, oracle conditions, multi-sigs) re-introduces most of the complexity that the fixed flavor scheme was chosen to avoid. A compromise: support a *bounded* predicate language in the circuit (time-lock + oracle-condition + up-to-3-sig) rather than a fully general one. The bounded language covers 90%+ of the programmable-predicate use cases while keeping the verifier compact.

19.5 Serialized B1 Sub-Notes (forward design)

Bearer (B1) notes invite micro-denomination use – walking-around money, tip jars – where one leaf per note is wasteful. The serialized design (*alberta-buck-notes-serialized*) packs N sub-notes under a single commitment leaf with a per-parent spent-bitmap on chain, amortising the mint cost across the family while keeping each sub-note independently spendable and double-spend-protected. It applies to B1 only (an addressed note’s recipient-targeted ciphertexts cannot be shared across siblings

without linking them). Designed but not implemented; the design memo carries the construction, the per-field security analysis, and the migration phasing.

19.6 Universal Spend Circuit (resolved for the note opening only)

All three flavours share one flavor-bound `spend.circom` note proof. The identity gate is not shared. A1 and A2 prove `deposit_fold_a1 / deposit_fold_a2`. B1 proves `verifyDepositorBinding` beside `identity_membership_b1`. The three entry points are `spendCoupled{A1,A2,B1}`.

19.7 Receiving Key, Still Open

The code chose one receiving key per wallet, rotated by the same counter as the accumulator salt. A key per payer, which would bound the damage of a leaked k , is not built. Guardian wrapping of a note payload is named as the recovery mitigation and is not a field of the payload. The value ciphertext is still encrypted under pk_{recv} ; whether the recipient alone needs to derive the face, given that the spend publishes it, is undecided. Trial decryption cost at realistic volumes has not been measured.

19.8 Regulatory Interaction with the Carrying Transfer

Jurisdictional regulators (FINTRAC, OSFI) have rules around bearer instruments and tax treatment of negative-interest accounts. BUCK's on-chain demurrage was argued through separately, and the Notes contract inherits that treatment with no pool-specific novelty – the contract is just another BUCK account.

The one new wrinkle is the Carrying-transfer egress itself: an ordinary transfer from a non-Carrying account hands the recipient "fresh" BUCK (the inflow ages from zero), whereas a Note redemption – a transfer from the `isCarrying` pool – apportions a share of the pool's accumulated buck-seconds onto the recipient. This means a Note recipient's post-redemption spendable value is less than their nominal balance – the difference being the demurrage they will pay on their next outgoing transfer. For tax and reporting purposes, whether the "face" or the "spendable" number should be used as the income figure is a question for tax counsel. The on-chain state (`balanceOf` and `buckSeconds`) is fully transparent either way.

20 Summary

BUCK Notes is the Alberta Buck's native private-bearer primitive. A single Merkle tree of commitments, a single nullifier set, and three realisable flavors – *A1* (addressed, public issuer; payroll and audited cheques), *B1* (bearer, public issuer; cashier's cheques), and *A2* (addressed, private issuer; the private-issuer self-wormhole) – span every use case the Alberta Buck series needs a bearer instrument to cover, from walking-around and cross-border targeted payments to cold-storage cryptographic notes. All three flavors – **A1**, **A2**, **B1** – are shipped, each with a third-party-free bilateral receipt. The addressed spends are ONE proof: `circuits/deposit_fold_a1.circom` and `deposit_fold_a2.circom` carry the reading relation, the account relation, the leaf that binds the two, its path, and – for *A2* – the issuer's own membership. The note $\leftrightarrow$ `eEnc` tie is a relation among them rather than a separate proof, because an equality inferred from two proofs that merely share a public point is not an equality. So "un-nameable is un-spendable" and "only M_{rec} can spend" hold against a colluding pair substituting ciphertexts (*Proofs*, Theorem 8). *A2* adds the key tie, so a minter cannot key one ciphertext to name its own Identity at mint and a registered puppet at spend (*Proofs*, Theorem 12). The fourth cell, private-bearer (*B2*), is ruled out by BUCK's mutual-decryptability invariant.

Demurrage needs no note-side machinery: the Notes Pool is the Notes contract's own ERC-20 balance – a regular BUCK account with its single packed `buckSeconds / timestamp` entry in the BUCK ledger. The only Note-side state beyond the Merkle tree and nullifier set is `noteFaceSum`, the total face value of live notes, kept as an audit scalar. Spend delivers the *full face value* to the recipient with an ordinary `transfer` that BUCK routes through its Carrying path (the pool is `isCarrying`), apportioning a share of the pool's accumulated buck-seconds onto the recipient. The recipient inherits the average pool BUCK-age and bears the resulting demurrage on their next standard outgoing transfer through the ordinary BUCK accounting path – exactly the demurrage they would have paid had they held the BUCK personally through the pool's residence period. The entire mechanism needs no new BUCK entry point – it reuses the `_carryingTransfer` path dispatched by the pool's `isCarrying` flag. Donations to the pool dilute its per-BUCK age and cheaply subsidise all subsequent redeemers. Conservation of nominal balance and Jubilee growth fall out exactly.

The addressed-versus-bearer distinction produces a sharp partition of which flavors are safe for long-term holding: *A1* and *A2* are cryptographically safe against third-party theft (only the recipient Identity M_{rec} can deposit, proven by the folded gate over (m_{rec}, k) and the receiving leaf, and – because authority is the *Identity*, not one account key – a lost account key is *recoverable* by re-registering a fresh account on the same M_{rec} ; only an explicit account-pinned opt-in treats key loss as terminal. *B1* is a civil-trust instrument: the issuer necessarily knows ρ , so *B1* cheques are meant to be cashed promptly and backed by the issuer's legal standing rather than by cryptography alone. This is an intrinsic property of the bearer predicate, not a policy choice.

The cryptographic architecture does not attempt to provide a pure-bearer note with no Identity hook. That use case remains the domain of physical trust models: tamper-evident seals over *B1* paper notes, issued by a trusted printer with a public warranty that ρ was erased after printing. This keeps the cryptographic/physical boundary explicit rather than contorting the cryptographic layer to approximate a property that inherently requires physical primitives.

A BUCK Note can live on chain as a commitment-opening pair, travel between wallets as a digital message, or be printed on paper as a QR-encoded slip with a hologram seal (for *B1*) or without one (for *A1*, *A2*). All three surfaces share the same cryptography and the same on-chain footprint. Counterfeiting is cryptographically impossible: a "fake" note's commitment is simply absent from the on-chain tree, the Merkle proof fails, and the wallet displays **FAKE**. Lost notes

are not extracted as seigniorage: the pool-age mechanism returns their value to the commons via demurrage pressure on subsequent live-note redemptions.

Issuance is on chain and identity-bound. Circulation breaks on-chain bulk traceability (no observable edge between payer and recipient) while preserving bilateral disclosure (both named parties always learn each other's M ; either is compellable on warrant). Deposit is identity-bound and auditable. This is stronger compliance than physical cash provides today, and stronger privacy than any CBDC has ever proposed.

No novel cryptographic assumption is introduced. Theorems 6 and 8 of the Proofs document (Proofs) apply to the A-spend predicate (the same Chaum-Pedersen re-encryption algebra, lifted to address the recipient Identity point M_{rec} and bound to registry membership); B-spend reduces to Schnorr signature verification plus a Schnorr PoK of the depositor's registered secret. The only new circuit-level primitive is Poseidon, which is already standard across the SNARK ecosystem. Implementation scope for Phases 1-4 (the public-issuer A1/B1 production surface, plus the shipped A2 issuer-binding apparatus) is 5-7 months for a focused team; the A2 recipient-key coupling that would make private-issuer notes collusion-resistant is a further reserved phase (in-SNARK registry set-membership over non-native BN254 $\mathbb{G}_1$).

BUCK Notes answers the question Chaum was asking in 1982: what does digital cash look like when you take "cash" seriously? The answer, forty years later: it looks like a commitment in an on-chain pool, openable as a message, a QR, or a printed slip, with a value that decays at the same rate your bank balance does, integrated with a complete monetary system that does not require a central bank to function. Alberta has, uniquely in the world, all four pieces required to build this. Whether to assemble them is, as ever, the only question that matters.